

COMPASS: Policy Learning and Semantic Synthesis for SMT Constraint Simplification

ANONYMOUS AUTHOR(S)

Constraint solving on complex satisfiability modulo theories (SMT) constraints such as nonlinear arithmetic, modular operations, and mixed logics remains challenging, as combinatorial blow-ups induce vast case splits and weak propagation under tight budgets. To address this challenge, we adopt a different approach: by concretizing a subset of the variables, we significantly reduce the complexity of the constraints. To efficiently identify the subset and its values, we present COMPASS, a solver-agnostic pre-solver that learns a policy for semantics-aware concretization. A reinforcement learning (RL) agent selects high-impact variables; a large language model (LLM) proposes semantics-guided values; the solver verifies each proposal in the loop. This decoupled design reduces branching and inter-variable coupling before the main solve and improves efficiency without modifying solver internals. Evaluations on SMTimer and SMT competition (SMT-COMP), covering quantifier-free linear integer arithmetic (QF_LIA) and quantifier-free nonlinear integer arithmetic (QF_NIA) with Z3, CVC5, MathSAT, and BVParti, show consistent gains. On SMTimer with Z3, the number of successfully solved instances rises from 72 to 94, a 30.6% increase, and the average time per solved instance drops from 561 s to 214 s, a 2.6× speedup. On QF_NIA, with the same 1200s budget, COMPASS enables Z3 to solve 324 instances that previously timed out with baseline Z3. These results indicate that learned, policy-driven, semantics-aware concretization can turn otherwise intractable formulas into solvable ones while preserving solver independence.

CCS Concepts: • **Software and its engineering** → **Software testing and debugging**; • **Theory of computation** → **Logic and verification**; • **Computing methodologies** → *Planning for deterministic actions*.

Additional Key Words and Phrases: Software Testing and Debugging, Logic and Verification, Planning for Deterministic Actions

ACM Reference Format:

Anonymous Author(s). 2018. COMPASS: Policy Learning and Semantic Synthesis for SMT Constraint Simplification. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 32 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

SMT solving is a fundamental technology in modern computer science, providing a core reasoning engine for diverse software engineering tasks, including software testing [?], program verification [7], and synthesis [?]. Despite the remarkable power of modern solvers like Z3 [24] and CVC5 [8], a fundamental and persistent challenge remains: the scalability bottleneck of SMT solving. This bottleneck becomes particularly acute when dealing with complex, real-world constraints involving non-linear arithmetic, high-dimensional variables, and mixed logics, frequently leading to timeouts and limiting the effectiveness of client applications [17].

To address this challenge, existing methods largely focus on two primary strategies. The first direction is *solver-level enhancement*, which focuses on improving the internal mechanisms of

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference acronym 'XX, June 03–05, 2018, Woodstock, NY

© 2018 ACM.

ACM ISBN 978-1-4503-XXXX-X/18/06

<https://doi.org/XXXXXXX.XXXXXXX>

the solver itself. This ranges from meta-level strategies like adaptive parameter tuning (e.g., Sym-Tuner [18]) to embedding learning directly into the solver’s core via neural branching heuristics [34]. The second strategy is constraint-level simplification, which operates on the formula itself before it is passed to the solver. This strategy encompasses a range of techniques that largely rely on fixed policies, from general-purpose syntactic methods like rewrite-based normalization [65], to sophisticated, specialized algorithms designed for specific theories like array constraints [? ?].

While each of these strategies has proven valuable, they possess fundamental limitations regarding the guidance gap—the lack of high-level, semantic insight to guide reasoning. Solver-level enhancement, while powerful, does not reduce the intrinsic semantic complexity of the constraints themselves and is often tightly coupled with solver internals. This leaves constraint-level simplification as the most direct approach to tackling a formula’s complexity. However, the techniques in this category, whether syntactic or specialized, are almost universally driven by fixed strategies and brittle, hand-crafted heuristics. They fundamentally lack a learned, adaptive policy to understand a formula’s semantic structure and make strategic, variable-level simplification decisions. This analysis reveals a clear and pressing research gap: a systematic, solver-agnostic method to learn where to simplify (*i.e.*, identifying high-impact variables) and how to simplify (*i.e.*, proposing semantically promising values).

To bridge this gap, we present COMPASS (Collaborative Method with Policy-learning And Semantic Synthesis)¹, a novel framework that reframes constraint simplification as a sequential decision-making problem. The core idea of COMPASS is to use a dual-agent AI architecture that performs strategic, variable-level concretization by leveraging the distinct strengths of each component. This architecture decomposes the complex simplification task: a strategic RL agent, whose policy is continuously adapted, decides what to simplify (variable selection), while a pre-trained LLM, serving as a stable source of general semantic knowledge, determines how to simplify (value suggestion). The RL agent’s policy is subsequently refined based on feedback gathered from this interactive process, which is quantified and guided by our hybrid reward system. This system provides a rich learning signal by combining sparse, ground-truth feedback from the SMT solver, dense heuristic guidance from pre-trained predictive models, and history-based penalties that discourage re-exploring failed paths. Through this feedback loop, COMPASS learns to recognize a formula’s semantic structure and applies policy-guided simplifications that significantly improve downstream solving efficiency.

To validate our approach, we evaluated COMPASS on QF_LIA and QF_NIA theories from SMTimer and SMT-COMP benchmarks, using four state-of-the-art solvers: Z3 [24], CVC5 [8], MathSAT [23], and BVParti [57]. COMPASS demonstrates consistent and substantial improvements. Specifically, on SMTimer with Z3, COMPASS increased the number of solved instances from 72 to 94 (+30.6%) and reduced the average solving time from 561s to 214s (2.6× speedup). The improvements extend across all tested solvers: CVC5 achieved a +57.9% increase in solved instances, MathSAT +244.8%, and BVParti +850%. Moreover, on the SMT-COMP benchmarks, COMPASS achieved a +4.6% improvement on QF_LIA, and on QF_NIA, it enabled Z3 to solve 324 instances that previously timed out within the same 1200s budget. **Yu: I think the result introduction should be consistent, e.g., COMPASS achieved a +4.6% (resp., +xxx%) improvement on QF_LIA (resp., QF_NIA).** These results demonstrate that policy-driven, semantics-aware concretization can turn otherwise intractable formulas into solvable ones while preserving solver independence.

The main contributions of this paper are as follows:

¹Analogous to a compass—where the needle (RL) sets the strategic direction and the dial (LLM) provides tactical detail—COMPASS integrates these roles to guide SMT solvers efficiently through vast and complex search spaces.

- We formalize SMT constraint simplification as a sequential decision-making problem and present COMPASS, a novel AI-assisted framework that employs a dual-agent architecture that decouples strategic variable selection (RL) from tactical value suggestion (LLM).
- We apply COMPASS to four existing SOTA solvers without modifying their internals. A complexity-guided, time-bucket-aware routing mechanism selectively invokes the AI-assisted simplification under tight budgets, enabling deployment-oriented cost–benefit trade-offs suitable for production environments.
- We conduct comprehensive evaluations demonstrating COMPASS’s effectiveness across challenging constraint sets and solver architectures. COMPASS achieves substantial improvements: 30.6% more solved instances and 2.6× speedup with Z3, with consistent gains across SOTA solvers. Ablation studies confirm that the RL-LLM synergy substantially outperforms individual components and baselines.

The remainder of this paper is organized as follows: Section 2 provides the necessary background. Section 3 presents the motivating example and formalizes the task as an Markov decision process (MDP). Section 4 details COMPASS. Section 5 reports our extensive experimental evaluation. Section 6 discusses limitations, and Section 7 reviews related work, before we conclude in Section 8.

Yu: Can be removed when exceeding page limit.

2 Background

2.1 Satisfiability Modulo Theories and the SMT-LIB2 Standard

Yu: no...we never explain challenge in background section. Maybe you can explain the basic syntax of SMT-lib2, since it is used in the example section. SMT is a fundamental decision problem for logical formulas that extends boolean satisfiability problem (SAT) with a combination of background theories, such as arithmetic, bit-vectors, and arrays [?]. This ability to reason about expressive, domain-specific constraints has made SMT a cornerstone of modern computer science, providing the core reasoning engine for diverse applications ranging from software verification and model checking [7] to automated test generation via symbolic execution [?]. The widespread success and practical impact of SMT have spurred the development of numerous highly-optimized solvers [8, 23, 24].

As a diverse ecosystem of powerful SMT solvers emerged, each with distinct specializations and performance characteristics, a critical need for a common interchange format arose. The SMT-LIB2 format [?] was established as the community’s solution, providing a unified and solver-agnostic standard. Its syntax is based on S-expressions, using a Lisp-like prefix notation where an operator precedes its arguments (e.g., `(+ x 5)` represents the expression $x + 5$). A problem definition in SMT-LIB2 is typically structured using a few core commands. The `declare-const` command, written as `(declare-const <symbol> <sort>)`, introduces a symbolic constant (i.e., variable) of a specified sort into the solver’s logical context. For example, `(declare-const x Int)` declares a symbolic integer named `x` that can be referenced throughout the formula. More generally, the `declare-fun` command, written as `(declare-fun <symbol> (<sorts>) <sort>)`, declares a function symbol, where `declare-const` serves as a convenient shorthand for declaring a function with no arguments. To manage complexity in formulas with deeply nested expressions, such as those presented in our motivating example (Section 3), the `let` construct, written as `(let ([<bindings>]) <term>)`, is essential. This construct allows one or more complex sub-expressions to be assigned to local variables within the `[<bindings>]` block for use within the body `<term>`, which significantly improves a formula’s readability and structure by avoiding repetitive sub-expressions. The `assert` command, written as `(assert <formula>)`, introduces a logical constraint—namely, the Boolean formula `<formula>`—into the solver’s current assertion stack. This stack accumulates all active

constraints, which are collectively evaluated when a satisfiability check is performed. The solver's primary goal is to determine if there exists a valid assignment for all declared constants that makes all asserted constraints simultaneously true, for which it will return one of three results: sat (satisfiable), unsat (unsatisfiable), or unknown (typically due to timeout or resource limits).

2.2 Reinforcement Learning for Sequential Decision Making

Yu: We discuss Reinforcement Learning for Sequential Decision Making. No need to discuss challenges in SMT query.

The outline could be: 1. What is Reinforcement Learning, the basic framework of RL (one or two paragraphs) 2. How to apply RL to a downstream task (one paragraph). Yu: What is MDP? It is used in the later section but never explained. introduce RL and MDP RL is a computational approach that addresses sequential decision-making problems by formalizing them as an interaction between two main components: a learning agent and its environment [60]. The agent learns to achieve a goal through a process of trial-and-error, where at each time step it observes the environment's state, selects an action, and receives a corresponding reward and the next state as feedback. In this paradigm, the environment is formally modeled as a MDP [?], which serves as the complete mathematical specification of the learning problem. A standard MDP is defined by a 4-tuple $(\mathcal{S}, \mathcal{A}, P, R)$, where $\mathcal{S}$ represents the set of all possible states, $\mathcal{A}$ represents the available actions, P is the state transition dynamics, and R is the reward function. The agent's fundamental task is then to learn a policy $(\pi : \mathcal{S} \rightarrow \mathcal{A})$ —a mapping from states to actions—that maximizes the cumulative reward it can collect.

For tasks with naturally bounded episodes, a finite-horizon MDP extends this formulation with an additional component, yielding a 5-tuple $(\mathcal{S}, \mathcal{A}, P, R, H)$, where H represents the finite horizon—the maximum number of time steps in each episode [60]. This finite-horizon constraint is particularly relevant for episodic tasks where each problem instance has a natural termination condition, such as reaching a goal state or exhausting available resources. In finite-horizon settings, the agent's objective becomes maximizing the expected cumulative reward within the bounded time horizon H .

explain how to apply RL to a real-world task Before an agent can learn such a policy to deal with a real-world task, the task's characteristics must be translated into the formal components of an MDP. This translation involves three key design choices: defining a state space ($\mathcal{S}$) that effectively captures all features of the task relevant for decision-making; defining an action space ($\mathcal{A}$) that encompasses all valid moves the agent can make; and designing a reward function (R) that provides feedback correctly aligned with the task's ultimate goal. For instance, in a classic grid-world navigation problem, an agent's state is typically its (x, y) coordinate, its actions are the set $\{\text{up, down, left, right}\}$, and its reward function might provide +1 for reaching a target and -1 for hitting an obstacle. Once a problem is framed in this way, a suitable RL algorithm can be applied, allowing the agent to learn a policy that navigates the grid efficiently.

explain the challenges of applying RL to a real-world task, particularly the reward function, correspond with the reward in methods While the MDP framework is conceptually straightforward, designing its components for complex tasks presents significant challenges, particularly the reward function. Feedback from the environment is often naturally sparse—meaning a non-zero reward is received only after a long sequence of actions. This sparsity leads to the difficult temporal credit assignment problem, making learning inefficient [60]. Algorithmic approaches such as Actor-Critic methods are designed to mitigate this issue [?]. They introduce a critic that learns a long-term value function, providing the actor (the policy) with a denser, internally-generated estimate of future rewards to guide its learning. However, since the critic's estimates are themselves ultimately derived from the sparse environmental feedback, this internal guidance can be slow to converge and

unreliable in highly complex settings, a well-known challenge in RL [?]. Consequently, effectively addressing severe reward sparsity often remains a fundamental problem in applied RL, frequently necessitating the design of problem-specific, hybrid reward systems that incorporate multiple, diverse signal sources [?].

2.3 Large Language Models for Semantic Reasoning

Yu: Usually introduce basic information about large language models. LLMs are deep neural networks, typically based on the Transformer architecture [?], that are pre-trained on vast amounts of text and code [?]. This extensive pre-training equips them with a powerful internal model of language, encompassing syntax, semantics, and common-sense knowledge. The primary operational paradigm for modern LLMs is *in-context learning* [?], where they can be steered to perform novel tasks based solely on the context provided in a carefully constructed prompt, without any changes to the model's parameters.

Beyond fluency in natural language, this pre-training has led to surprising *emergent abilities* [?], with recent work demonstrating LLMs' growing capacity for formal and heuristic reasoning in tasks such as invariant generation and logical problem solving [30, 48]. A key manifestation of this semantic reasoning is semantic synthesis—the ability to not just understand a context, but to actively generate a plausible continuation or structured output that adheres to its semantic constraints. It is this capability that allows an LLM to function as a powerful engine for proposing candidate solutions that are consistent with a given problem's requirements.

3 Motivation and Problem Formulation

In this section, we first present a synthetic but representative motivating example that illustrates the solver bottleneck and the core intuition behind strategic simplification. Then, we formalize the simplification task as an MDP suitable for learning-based methods.

3.1 Motivating Example

To concretize the challenge of semantically blind heuristics and to illustrate why the expert-like intuition introduced in the introduction is effective, we use a synthetic but representative motivating example that captures the challenging path conditions commonly encountered in program analysis.

3.1.1 A Challenging SMT Formula. Figure 1 presents a synthetic SMT formula in SMT-LIB2 format designed to be representative of the challenging path conditions observed in program analysis [17]. The formula consists of six integer variables (x, y, z, a, b, c) and four conjunctive constraints expressed as separate assert statements, each containing complex nested expressions with let bindings for intermediate computations.

This formula exhibits several sources of complexity that are notoriously difficult for SMT solvers. First, the nested let bindings create deep expression trees with high-degree nonlinear polynomials (e.g., $(^ x 4)$ for x^4) and complex products such as $(^ (^ (+ z 15) (- z 30)) (- z 11))$, which enlarge the search space into vast, nonconvex regions, reflecting the intrinsic hardness of nonlinear integer programming [44]. Second, modular arithmetic operations (mod function) introduce discontinuities that force solvers to perform extensive case splits, frequently causing exponential blow-up. Third, the extensive use of type conversions (to_real) between integer and real domains adds additional complexity layers. Finally, variables are strongly coupled across multiple, interdependent assert statements; a change to one variable propagates throughout the formula, undermining local propagation and pruning. As a result, even a state-of-the-art solver like Z3 [24] fails to return a result within a 1200-second timeout according to our study (Section 5), exemplifying the solver bottleneck that motivates our approach.


```

246 (assert (let ((?x34 (to_real c))) (let ((?x32 (to_real b))) (let ((?x30 (^ a
247 2))) (let ((?x28 (^ z 2))) (let ((?x24 (^ y 3))) (let ((?x22 (+ (mod (+ (* x
248 37) 23) 101) (mod (* (- x 30) (- x 30)) 57)))) (= (+ (- (+ (- (+ (to_real
249 ?x22) ?x24) ?x28) ?x30) ?x32) ?x34) 150.0))))))
250 (assert (let ((?x47 (^ c 2))) (let ((?x32 (to_real b))) (let ((?x45 (- (+ (^ x
251 4) (^ y 3)) (to_real (* (* 5 z) a)))) (< (- (+ ?x45 ?x32) ?x47) 400.0))))
252 (assert (let ((?x87 (+ (mod (+ (* a 12) 301) 101) (mod (* (- a 15) (- a 30))
253 57)))) (let ((?x93 (+ ?x87 (mod (* (* (+ a 15) (- a 30)) (- a 11)) 9))))
254 (let ((?x78 (to_real (* b c)))) (let ((?x75 (to_real a))) (let ((?x64 (+
255 (mod (+ (* z 12) 301) 101) (mod (* (- z 15) (- z 30)) 57)))) (let ((?x72 (+
256 ?x64 (mod (* (* (+ z 15) (- z 30)) (- z 11)) 9)))) (let ((?x52 (^ y 2)))
257 (let ((?x51 (^ x 2))) (let ((?x53 (- ?x51 ?x52))) (> (- (+ (+ ?x53 (to_real
258 ?x72)) ?x75) ?x78) (to_real ?x93))))))
259 (assert (let ((?x78 (to_real (* b c))) (let ((?x75 (to_real a))) (let ((?x101
260 (+ (mod (+ (* z 37) 23) 101) (mod (* (- z 30) (- z 30)) 57)))) (let ((?x52
261 (^ y 2))) (let ((?x51 (^ x 2))) (let ((?x53 (- ?x51 ?x52))) (and (distinct
262 (- (+ (+ ?x53 (to_real ?x101)) ?x75) ?x78) 55.0) true))))))

```

Fig. 1. An illustrative SMT formula with nonlinear integer constraints. There are differences between the previous SMT example and the actual implemented document; further adjustments will be made subsequently.

3.1.2 Guided Simplification via Variable Concretization. We find that an efficient way to solve the formula is to simplify it by first identifying and then concretizing the most influential variable. To select the variable, we employ a heuristic analysis based on three metrics: frequency (F) measures the total number of times a variable appears across all assert statements; complexity (C) counts involvement in high-cost nonlinear operations (e.g., $\wedge$ for powers, nested multiplications, mod operations); and coupling (P) represents the number of distinct assert statements in which the variable participates. A higher total “Impact Score” ($I = F + C + P$) indicates a greater potential for simplification. Applying this analysis to our example, variable z emerges as the most influential with the highest Impact Score of 11 (i.e., $4 + 3 + 4 = 11$), as it appears in all four assert statements and is involved in the most complex nested expressions within the let bindings. This makes it the prime candidate for concretization. Detailed per-variable scores are provided in Appendix C.

Once the candidate variable z has been identified, the next challenge is to choose an appropriate value for concretization. An incorrect choice may render the formula unsatisfiable, in which case we cannot determine whether it is genuinely unsatisfiable or merely appears so due to the wrong concretization. Human experts typically draw on a set of problem-solving heuristics common to software testing and formal verification. For this formula, two heuristics are particularly effective: (i) identity-element substitution, which replaces variables with neutral (identity) values to simplify algebraic expressions; and (ii) boundary-value analysis, which prioritizes small, simple integers (e.g., 0, 1, -1). The expert applying these strategies would first recognize that z ’s primary contribution to intractability stems from its role within multiple, nested mod expressions in the third and fourth assert statements that introduce severe discontinuities. The strategic goal is thus to select a value that resolves these complex let-bound expressions. While both 0 and 1 are simple boundary values, the multiplicative identity, 1, is the superior heuristic choice here. This choice surgically simplifies the formula: it collapses the complex modulo terms (e.g., $(\text{mod } (* (* (+ z 15) (- z 30)) (- z 11)) 9)$ becomes a constant 5 when $z = 1$) while preserving crucial algebraic structure in the let bindings. Terms like $(\wedge z n)$ reduce to 1 and $(* z k)$ to k , in contrast to the additive identity, 0, which would nullify entire sub-expressions and risk over-constraining the problem. With the

intractable complexity originating from z eliminated, Z3 finds a satisfying model in under one second on the same machine used in Section 5.

3.1.3 Limitations of Static Heuristics. *Yu: to check.* The introduction to the method has been revised to align with the changes made in the intro. The example demonstrates the advantage of choosing both an appropriate variable and value for formulas with complex let bindings. However, this success depends on an analysis tuned to the formula’s nested structure: selecting which heuristic to apply and how to balance their trade-offs is highly context-dependent and lacks a general, automated procedure. Specifically, such heuristics, including the “Impact Score” we introduced, are not general: they rely on superficial proxies (e.g., occurrence counts or local operator weights) rather than the semantics of how assignments reshape coupled assert statements. Although the “Impact Score” correctly identified z as a critical variable by analyzing its presence across multiple let bindings, it gives no guidance for choosing a value. The effectiveness of setting $z = 1$ arises from its dual effect: it removes mod-induced discontinuities while preserving essential algebraic structure in the nested expressions. Because value selection is constraint-specific and depends on the intricate structure of let bindings, designing a single, static and general heuristic is infeasible.

These limitations indicate that a “one-size-fits-all” heuristic is insufficient. Indeed, recent work shows that adaptive, learning-based methods often outperform fixed heuristics on structurally diverse reasoning tasks [20, 39, 64, 67]. Motivated by these findings, our framework introduces a novel approach that, instead of learning a single monolithic policy, decomposes the simplification task into two complementary roles. We introduce a synergistic approach where RL agent learns a strategic policy to select what variable to simplify, and an LLM performs the tactical, semantic task of proposing how to assign its value. This synergy allows each component to leverage its unique strengths—the RL agent for long-term, feedback-driven planning and the LLM for rich, context-aware reasoning—enabling our system to navigate the complex semantic interdependencies of SMT formulas.

3.2 Problem Formulation

This section formalizes our approach. We first define the properties of an ideal simplification, establishing the core objectives of our task. We then model this task as a finite-horizon Markov Decision Process, providing the mathematical foundation for our learning-based solution.

3.2.1 The Optimal Simplification Problem. We define the problem of simplifying an SMT query q by strategically replacing some of its symbolic variables V with concrete values—a process called concretization. A concretization action is a pair $a = (v, c)$ for a variable $v \in V$ and a concrete value c .

Problem 3.1 (Optimal Query Simplification). Given an SMT query q and a time budget T_{budget} , find a sequence of concretization actions $\sigma = \langle a_1, \dots, a_k \rangle$ that transforms q into q' such that:

- (1) Efficiency: The solving time of the simplified query is minimized: $T_{solve}(q') \leq T_{budget} \wedge T_{solve}(q') \leq T_{solve}(q)$, where $T_{solve}(q)$ is the time cost for solving a SMT query q .
- (2) Soundness: If the simplified query q' is satisfiable, then the original query q is satisfiable (i.e., $q' \Rightarrow q$).

The two properties capture the core requirements of practical query simplification. *Efficiency* ensures that the method is useful in practice: the simplified query q' must not only respect the time budget T_{budget} but also be easier to solve than the original query q . Without this property, simplification could waste computational effort or even make solving harder. *Soundness* guarantees correctness: since concretization strengthens the constraints by fixing variable values, any satisfiable solution to q' must also be a solution to the original query q . However, we do not require completeness—there

is no guarantee that satisfiability of q implies satisfiability of any concretization q' , since an unfortunate choice of concretized values may eliminate valid solutions. This trade-off is fundamental: we gain efficiency and soundness, but accept potential loss of completeness.

3.2.2 Modeling as a Markov Decision Process. Finding an optimal sequence σ is intractable due to the combinatorial search space, making the problem well-suited for a learning-based approach. To learn an effective simplification strategy, we instantiate the finite-horizon MDP framework introduced in Section 2. Since each simplification attempt consists of a finite sequence of actions with a natural termination condition (when all variables are concretized or the time budget is exhausted), we model the task as a finite-horizon MDP, where each attempt constitutes an *episode*. Following the 5-tuple formulation from Section 2, our MDP is defined as:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, H),$$

with:

- **State Space ($\mathcal{S}$):** A state $s_t \in \mathcal{S}$ is a tuple (q_t, τ_t) , where q_t is the current SMT formula and τ_t is the *history* of concretization actions applied so far. In practice, s_t is represented by a high-dimensional vector encoding of q_t that captures its semantic properties (Section 4.3).
- **Action Space ($\mathcal{A}$):** An action $a_t = (v_t, c_t)$ in the environment is a pair consisting of a variable v_t and a concrete integer value c_t . This forms a vast composite discrete action space.
- **Transition Function (P):** State transitions are deterministic. Applying an action $a_t = (v_t, c_t)$ to a state $s_t = (q_t, \tau_t)$ yields a new state $s_{t+1} = (q_{t+1}, \tau_{t+1})$, where $q_{t+1} = \text{SUBSTITUTE}(q_t, v_t, c_t)$ and τ_{t+1} is the history τ_t appended with a_t .
- **Reward Function (R):** The reward $r_t = R(s_t, a_t)$ is generated by our hybrid reward system (Section 4.4), which combines sparse ground-truth signals, dense predictive guidance, and history-based penalties.
- **Horizon (H):** The maximum length of each episode is a fixed, finite horizon H , which we set to the total number of variables in the initial formula, $|\mathcal{V}|$.

In COMPASS, the RL agent’s task is to learn the strategic component of this process. Therefore, the agent learns a policy $\pi : \mathcal{S} \rightarrow \mathcal{V}$ that maps a formula’s state to the selection of a high-impact variable. The LLM then provides the tactical component by proposing a value for the selected variable, thus completing the full action a_t . By maximizing the expected cumulative reward within the horizon H , the RL agent learns to guide the simplification process toward short and effective solution paths.

4 Methodology

This section presents the design and implementation of COMPASS. We begin by providing a high-level overview of the framework’s architecture (Section 4.1). We then detail the core stages of our simplification pipeline: the variable normalization process that canonicalizes input formulas (Section 4.2), the feature encoding and predictive model training that constructs the agent’s state representation (Section 4.3), the multi-level hybrid reward system (Section 4.4), and our dual-agent policy formulation that synergizes RL and LLMs (Section 4.5). Finally, we detail the simplification algorithm (Section 4.6).

For clarity of terminology, we use “semantics-aware” to denote the policy-learning paradigm and reasoning that leverages program semantics during variable-level concretization, and “semantics-guided” to denote the LLM-driven value proposal subroutine whose candidate values are guided by semantic cues. **Yu: Overall comment: I think lots of contents are generated by llm. you should write your on own. LLMs are only used for polishing.**

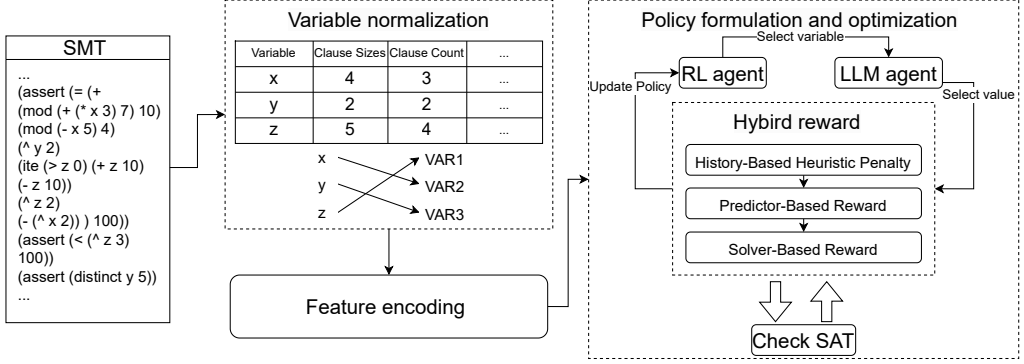


Fig. 2. Workflow of COMPASS.

Yu: The content before sec 4.1 is too long, usually we only illustrate what this section do, e.g., “In this section, we present our methodology, in particular, we discuss the major components including deoptimization, performance checking and test case reduction in detail.”

However, the last paragraph (start with "for clarity..." is ok.)

4.1 Overview

Figure 2 illustrates the architecture of COMPASS which consists of four components: variable normalization, feature encoding, hybrid reward system, and dual-agent simplification policy. The process begins with variable normalization which creates a consistent foundation for learning. To ensure our models can generalize across formulas that are structurally identical but use different variable names, this stage canonicalizes the input by sorting variables based on a stable vector of their structural attributes, yielding a permutation-invariant representation for the agent.

The canonical formula is then transformed into a state representation by our feature encoding component. This component converts the symbolic text into a high-dimensional vector designed to capture the formula's underlying structure. This vector serves two primary purposes. First, it provides the contextual input required for the RL agent's policy decisions. Second, it is used to train two predictive models: a classifier to estimate a formula's solvability and another to predict its solving time. As we will see, these pre-trained predictors are crucial for guiding the agent's learning.

To guide the learning process effectively, we then designed a hybrid reward system. Recognizing that feedback in the SMT environment is inherently sparse and expensive, this system provides a rich learning signal by combining sparse but definitive outcomes from the SMT solver, dense heuristic guidance from pre-trained predictive models, and history-based penalties to prune the search space.

Finally, guided by this rich reward signal, the heart of our approach is the dual-agent architecture. This architecture is designed to address the challenge of learning over the environment's vast, composite discrete action space, where a complete action consists of both a variable and a value. Instead of learning a single, monolithic policy to navigate this space, our solution decomposes the decision-making process into two complementary roles. A strategic RL agent learns a policy to select what to simplify (variable selection), effectively deciding which dimension of the action space to explore. Subsequently, a tactical LLM uses its semantic reasoning capabilities to propose *how* to simplify (value generation), completing the action pair. This division of labor allows each component to leverage its unique strengths—the RL agent for long-term, feedback-driven planning and the LLM for rich, context-aware reasoning.

Table 1. Attributes for variable normalization, ordered by importance.

Attribute	Description
Variable clause sizes	The sum of elements in clauses where the variable appears.
Variable to clause count	The number of distinct clauses the variable appears in.
Variable frequencies	The total number of times the variable occurs in the formula.
Total logic operations	The number of logical operations the variable participates in.
Variable constant interactions	The count of clauses where the variable appears with a constant.

4.2 Variable Normalization

Yu: This subsection seems not clear to me. The method should be explained more clear (discuss steps first, then explain the benefit.). e.g., adding examples. Second, the design choice is missing (e.g., why do you use the canonicalization method). Finally, why does this method work?

A foundational challenge for applying machine learning methods to symbolic formulas is input variance [10, 72]: two structurally isomorphic formulas may be textually distinct due to arbitrary variable naming, which prevents a learned model from generalizing. To overcome this, COMPASS begins with a deterministic canonicalization of the input SMT formula. The goal of this process is to standardize the naming and ordering of variables based on their structural roles, ensuring that any two structurally equivalent formulas are mapped to the exact same textual representation. This allows our learning models to focus on the underlying semantic structure rather than superficial naming conventions.

4.2.1 Canonicalization via Structural Sorting. The core of this process is a deterministic sorting process that reorders variables based on their structural importance. For each variable, we first extract a vector of numerical attributes that serve as proxies for its influence within the formula. These attributes, detailed in Table 1, are arranged in a predefined order of importance, including metrics like the total size of clauses a variable appears in, its frequency, and the number of distinct clauses it participates in. The sorting itself is performed using this ordered vector of attributes. Specifically, all variables are first sorted according to the most important attribute (i.e., ‘Variable clause sizes’). Any ties among variables are then resolved by sorting them according to the second most important attribute (‘Variable to clause count’), and this tie-breaking process continues sequentially down the entire attribute list. This stable, multi-level sorting procedure guarantees that a unique and deterministic ordering is produced for any given formula. Finally, the sorted variables are systematically renamed to a canonical form, such as $VAR_1, \dots, VAR_n$, completing the canonicalization.

4.2.2 Impact on Policy Learning. A simpler approach, like alphabetical sorting, would be brittle and fail to capture any structural information. In contrast, our chosen attributes act as robust proxies for a variable’s semantic importance, a principle that aligns with feature engineering for code analysis [29]. By encoding this importance into the variable ordering, we create a representation that is not only consistent but also informative for the downstream agent. This method is effective because it makes the learning models invariant to superficial syntactic differences, a crucial step when applying machine learning to source code [2]. This process significantly reduces the variance of the state space the RL agent must explore [11], provides a stable action space where an action consistently targets a variable of similar structural importance, greatly simplifying the policy learning challenge [35].

4.3 Feature Encoding

introduce "what" in overview, introduce "why" in this section with short explanation To enable an RL agent to make informed decisions, the SMT formula must be transformed into a rich, machine-readable state representation. It is well-established that purely syntactic representations often fail to capture the complex semantic relationships within structured code or logical formulas, limiting the generalization capabilities of learning models [2]. We therefore employ a pretrained language model (PLM) to serve as our feature encoder, a technique that has proven highly effective for generating deep, contextualized representations of source code [25].

4.4 The Hybrid Reward System

Effective policy learning in RL hinges on receiving frequent and informative feedback. However, the SMT solving environment presents two fundamental challenges for reward design. First, the SMT solver is a well-known performance bottleneck [17]; relying solely on its ground-truth outcomes would require invoking it after every action, rendering the learning process computationally intractable. Second, the environment's rewards are naturally sparse. An agent may perform a long sequence of simplification actions before receiving a definitive outcome, a classic challenge in RL known as the temporal credit assignment problem [60].

To overcome these challenges, we designed a hybrid reward system that synergistically combines signals from two distinct real-time sources—the solver and a set of predictive models—and augments them with a history-based heuristic penalty. Crucially, to balance the trade-off between signal density and computational cost, the system employs a partial evaluation strategy, where feedback is periodically generated from a random subset of the formula's constraints rather than the entire set.

4.4.1 Solver-Based Reward. The most definitive feedback originates from the SMT solver, providing a high-fidelity, ground-truth signal. The solver's primary role is to provide the ultimate reward through a comprehensive check on the entire formula, typically triggered once at the end of an episode. This anchors the policy to the true objective but is inherently sparse. To enrich the learning process with more frequent feedback, we also employ the solver strategically for partial evaluations. These lightweight checks involve invoking the solver with a short timeout on a subset of constraints, from which a quick unsat result yields a medium-fidelity negative reward, enabling efficient pruning of the search space at a much lower computational cost.

4.4.2 Predictor-Based Reward. Complementing the solver, our pre-trained predictive models offer a continuous stream of low-cost, dense guidance. Specifically, we employ two distinct neural network models: a binary classifier, P_{solve} , that predicts the likelihood of a formula being satisfiable, and a multi-class classifier, P_{time} , that estimates the solving time by assigning it to a predefined complexity bucket.

To ensure the integrity of our evaluation and prevent any data leakage, these predictors are trained on a separate partition of the data. For each benchmark dataset used in our experiments, we perform a strict 50/50 split: one half is used exclusively for training P_{solve} and P_{time} , while the other half is reserved as the test set for our online simplification agent.

During the simplification process, these lightweight models provide the reward signal in two primary ways. They are tightly integrated with our partial evaluation strategy, where they are queried on the same constraint subset as the solver to provide an immediate estimate of progress. Furthermore, their low computational cost allows them to be queried on the full formula at any step, ensuring a constant stream of moment-to-moment feedback. This dense signal directly mitigates the temporal credit assignment problem and provides a smooth learning gradient for the agent.

4.4.3 History-Based Heuristic Penalty. In addition to the two real-time reward sources, our system incorporates an orthogonal component that leverages the agent’s past experience: a history-based heuristic penalty. This mechanism functions as a form of agent memory, utilizing a repository of counterexamples that stores previously failed action sequences. If the agent’s current action sequence matches a known failing path, a significant negative penalty is immediately applied. This low-cost lookup operation acts as an efficient search-space pruning mechanism and provides a deterministic penalty for repeated mistakes, thereby accelerating policy convergence.

4.4.4 Synergy. The final reward signal r_t is a synergistic composition of these components. The definitive but sparse signals from full solver evaluations ensure the policy converges towards the true objective. Meanwhile, the combination of denser feedback from partial solver checks and continuous heuristic guidance from the predictors provides a rich gradient for exploration. Finally, the history-based penalty leverages memory to efficiently prune the search space. Together, they form a robust and efficient hybrid reward function capable of guiding the agent through the challenging landscape of SMT constraint simplification.

4.5 Dual-Agent Architecture

To learn an effective simplification strategy, we formalize the task as a MDP and train a RL agent. The agent’s goal is to learn a policy, $\pi(a_t|s_t)$, that maps a given formula state to a simplification action that maximizes the expected long-term reward. This section details our complete MDP formulation by defining its core components—the state representation (s_t), the action space (a_t), and the hybrid reward (r_t)—and concludes by detailing the approach for optimizing the agent’s policy.

4.5.1 State Representation. The agent’s decision at each timestep is conditioned on the current state of the formula, s_t . As defined in Section 4.3, we use the high-dimensional vector embedding produced by a PLM as the state representation. This vector captures the rich semantic and structural properties of the canonicalized formula, providing the necessary context for the policy.

4.5.2 Action Space and Dual-Agent Design. The action space for simplification is a complex, hybrid domain $\mathcal{A} = \mathcal{V} \times \mathcal{C}$, where $\mathcal{V}$ is the finite, discrete set of canonical variables, and $\mathcal{C}$ is the virtually infinite set of possible integer values. A single RL agent cannot tractably learn a policy over this joint space. Our core contribution is to decompose this action into two sequential sub-problems, governed by the principle of task decomposition based on the core strengths of each component[32]. The high-level, strategic decision of *what* variable to simplify is assigned to an RL agent, whose policy operates over the finite space $\mathcal{V}$. Conversely, the low-level, tactical decision of *how* to simplify is delegated to a pre-trained LLM. Conditioned on the state and the chosen variable, the LLM generates a contextually plausible value. This is achieved by providing it with a carefully constructed prompt that includes the full formula context, the target variable, and a list of previously failed values (counterexamples) to steer it away from known unsuccessful attempts, as shown in Figure ?? . A key consequence of this design is that only the RL agent undergoes online, gradient-based policy updates, while the LLM remains a static, pre-trained model with frozen parameters, adapting only through the in-context information provided in the prompt. This paradigm of pairing a dynamic, learning policy with a static, pre-trained model has proven effective for training large models with reinforcement learning. For instance, Ouyang et al. [46] use a frozen reference model to regularize the policy during optimization, ensuring its core capabilities are preserved. In our framework, the LLM serves as a stable source of general semantic knowledge, while the RL agent specializes in learning a policy for the specific task of SMT simplification. This is intentional, as it preserves the specialization of each agent. The RL agent’s role is to adapt and learn a domain-specific strategy,



System

You are an advanced SAT/SMT solver, focusing on the optimization and resolution of logical constraint problems. Your input consists of two parts: first, the counterexamples of failed solution assignments previously chosen, and second, the strings in SMT-LIB format that needs to be solved. You should analyze these inputs, using logical reasoning and heuristic methods to determine which variable assignments led to the failure of the solution, and identify the variable assignments that satisfy all constraint conditions. The output should be a set of specific variable assignments that can satisfy all the constraints defined in the strings. Your task is to find the specific values that should be assigned to the variables provided in the prompt to ensure that the entire constraint system is satisfiable. You should output only the numeric value, with an example as follows: `<value>`. Do not output any other text, explanations, or symbols.



User

Here is the counterexamples of failed solution assignments previously chosen in json formats: `$ce_json$`. Here is the SMT file content: `$smtlib_str$`.

This is the The variable values from the previous failed SAT solving attempt and SMT text given to you in segments; analyze it. To speed up the solution and obtain a SAT result, provide a specific number that `$variable_pred$` should be assigned to. However, do not choose the values that have already failed to solve. Output only the numeric value. Do not output any other text, explanations, or symbols. The output must be a single number.



Output example

12

Fig. 3. Prompt template for heuristic value generation by the LLM.

while the LLM's role is to serve as a stable source of general semantic knowledge. Attempting to fine-tune the LLM on the sparse and noisy reward signal from the solver would risk catastrophically degrading its valuable pre-trained abilities and would be computationally infeasible. While the LLM does not learn via gradients, it does adapt within an episode through in-context learning: by incorporating previously failed values into its prompt as counterexamples, it is steered away from known unpromising suggestions.

4.5.3 Hybrid Reward. To guide the agent's learning, we use the multi-level hybrid reward system detailed in Section 4.4. The final reward, r_t , for taking a complete action $a_t = (v_t, c_t)$ is a composite signal combining sparse, ground-truth feedback from the solver with dense, estimated rewards from our predictive models. This hybrid reward is crucial for mitigating the credit assignment problem. We emphasize that the LLM is part of the action-generation process, not the reward-generation process.

4.5.4 Policy Optimization via Soft Actor-Critic. We learn the variable selection policy π using soft actor-critic (SAC) [27], an off-policy actor-critic method based on the maximum entropy framework. Its selection is principally motivated by two features that directly address the challenges of SMT constraint simplification. First, interacting with the environment is computationally expensive due to the cost of SMT solver invocations. As an off-policy method that leverages a replay buffer, SAC is highly sample-efficient, making it well-suited for learning in environments where data collection is costly [36]. Second, the space of effective simplification strategies is vast and requires extensive exploration to avoid converging to a suboptimal policy. The core mechanism of SAC, entropy maximization, encourages robust exploration by incentivizing the agent to act as randomly as possible while still accomplishing its objective, a critical feature for complex decision-making problems [27]. The SAC agent consists of an actor network, which models the policy, and critic networks, which estimate the expected future rewards. The learning process follows a standard online RL loop. At each step, the agent observes state s_t , selects a variable v_t , the LLM proposes a value c_t , and the environment returns the hybrid reward r_t and the next state s_{t+1} . This transition (s_t, v_t, r_t, s_{t+1}) is stored in a replay buffer. The agent is then trained by sampling mini-batches from this buffer to update its actor and critic networks, progressively improving its policy to maximize the expected cumulative reward.

4.6 Overall Algorithm

Yu: "Overall Algorithm" seems to be a better title of the subsection

Yu: give the high-level idea of the algorithm (e.g., explain the alg using three major steps), then explain the algorithm line by line. The individual components of our methodology are orchestrated by the COMPASS algorithm, a single, closed-loop procedure that operationalizes our framework, as detailed in Algorithm 1. The algorithm follows a strategic, trial-and-error exploration process structured as a nested loop: an outer loop initiates new simplification attempts (“trials”), while an inner loop extends the current simplification sequence step-by-step. A key innovation guiding this process is a performance-monitoring mechanism that defines the “error” phase of this exploration: it uses our predictive models to greedily pursue promising paths while quickly abandoning those that degrade performance. These abandoned sequences are treated as learning opportunities—“errors”—and are recorded as counterexamples to inform subsequent trials. The process begins by preparing the initial formula Q_{init} through variable normalization and initializing the SAC agent and the repository for these counterexamples (Lines 2–4). The outer loop then manages the overall simplification process, launching new trials until a satisfying assignment is found or an external termination condition (e.g., a timeout) is met (Line 5).

Each trial starts by resetting the environment to the original, canonicalized formula, Q_{base} (Line 6). The feature encoding component (Section 4.3) is invoked to generate the initial state embedding, s , and a baseline performance score, $last_perf$, is initialized to zero (Line 7). This provides a neutral starting point for performance comparison throughout the trial. The inner loop (Line 9) then begins to extend the current simplification sequence. At each step, the dual-agent policy (Section 4.5) is executed: the RL agent selects a variable v_t , and the LLM proposes a value c_t based on the accumulated counter-examples from previous failed trials (Lines 10–11).

After the formula is simplified, the framework evaluates the new state. The feature encoder is re-invoked to produce the next state embedding s' (Line 14), and the backend solver attempts to solve the modified formula. The multi-level hybrid reward computation simultaneously evaluates both the reward signal and a performance score, $current_perf$, based on predictive model outputs and solver results (Line 16). This performance check is the primary driver of the exploration strategy. If the score degrades compared to the baseline (Line 18), the current trial is deemed a failure. The failing sequence is recorded (Line 19), and the inner loop terminates, triggering a new trial. Otherwise, the agent’s policy is updated using the full multi-level hybrid reward (Line 16), as detailed in Section 4.4, and the state is advanced for the next iteration (Line 24).

5 EVALUATION

This section empirically evaluates our framework, COMPASS. We designed our experiments to answer three core research questions that progressively validate our approach, moving from overall effectiveness to internal component analysis and finally to practical deployment.

5.1 Research Questions

Our evaluation aims to answer the following research questions:

RQ1. Effectiveness of COMPASS. How effective is COMPASS across different constraint types, and SMT solver architectures?

RQ2. Ablation Study of COMPASS. What are the individual contributions of the RL and LLM components, and how do different LLMs affect overall performance?

RQ3. Effectiveness of Selective Simplification. Does a selective simplification strategy for the COMPASS improve the overall trade-off between performance and computational cost compared to a non-selective approach?

Algorithm 1 The COMPASS Simplification Algorithm with Performance-Monitored Exploration**Input:** Q_{init} - initial SMT constraint formula; S - backend SMT solver

```

1: procedure SIMPLIFYCONSTRAINT( $Q_{init}, S$ )
2:    $Q_{base} \leftarrow \text{NORMALIZE}(Q_{init})$ 
3:    $agent \leftarrow \text{INITIALIZE\_SAC}(\text{GETACTIONSPACE\_SIZE}(Q_{base}))$ 
4:    $counter\_ex \leftarrow []$  ▷ Stores all discovered failing sequences
5:   while True do ▷ Outer Loop: Manages new trials until solution or termination
6:      $Q \leftarrow Q_{base}; s \leftarrow \text{EMBED}(Q)$ 
7:      $last\_perf \leftarrow 0$  ▷ Initialize performance baseline for the trial
8:      $current\_sequence \leftarrow []$ 
9:     while True do ▷ Inner Loop: Extends the current simplification sequence
10:       $v_t \leftarrow agent.SELECTVARIABLE(s)$ 
11:       $c_t \leftarrow LLMPROPOSEVALUE(Q, v_t, counter\_ex)$ 
12:       $Q' \leftarrow \text{CONCRETIZE}(Q, v_t, c_t)$ 
13:       $current\_sequence.APPEND((v_t, c_t))$ 
14:       $s' \leftarrow \text{EMBED}(Q')$ 
15:       $(res, t_{solve}) \leftarrow S.SOLVE(Q', \text{GETADAPTIVE\_TIMEOUT}(s'))$ 
16:       $(r, current\_perf) \leftarrow \text{COMPUTE\_HYBRID\_REWARD}(res, t_{solve}, s, s', counter\_ex)$ 
17:       $agent.UPDATE(s, v_t, r, s')$ 
18:      if  $current\_perf < last\_perf$  then
19:         $APPENDTOHISTORY(counter\_ex, current\_sequence)$  ▷ Record failing sequence
20:        break ▷ End this trial and start a new one
21:      else if  $res = sat$  then
22:        return  $\langle sat, \text{GETMODEL}(S) \rangle$ 
23:      end if
24:       $s \leftarrow s'; Q \leftarrow Q'; last\_perf \leftarrow current\_perf$  ▷ Advance to the next step
25:    end while
26:  end while
27:  return  $\langle unknown \rangle$ 
28: end procedure

```

Our evaluation is designed to provide a comprehensive validation of COMPASS. We begin by addressing the foundational questions of overall effectiveness (**RQ1**) and the synergistic contributions of the core components through ablation studies (**RQ2**). Building on this foundation, **RQ3** addresses the challenge of practical deployment. While COMPASS is powerful, its AI-driven process introduces computational overhead compared to direct solving. Naively applying this expensive simplification to every incoming constraint would be inefficient, potentially slowing down the overall system on simpler problems where it is not needed. Therefore, a mechanism to selectively apply it only to genuinely difficult cases is essential. RQ3 is thus designed to investigate this practical trade-off, determining whether the performance benefits gained by applying COMPASS to hard problems can justify and outweigh the computational overhead introduced by the guidance mechanism itself.

5.2 Experimental Setup

5.2.1 Implementation. Our implementation is built upon the Pearl framework² and utilizes Ollama³ for language model hosting. The complete implementation comprises approximately 20,150 lines of Python code for core experimental functionality, including constraint solving orchestration, multi-solver integration, experimental evaluation framework, and data processing utilities.

5.2.2 Benchmarks. Our evaluation is conducted on two distinct benchmarks: SMTimer[39], which contains program-derived constraints from Coreutils⁴, BusyBox⁵, angr⁶, and KLEE, and the public SMT-COMP⁷ benchmarks, which we use to assess scalability against harder, multi-theory constraints[9, 16, 40]. To ensure a fair and meaningful comparison, we apply a consistent preprocessing pipeline to these datasets. This involves: (i) excluding trivially solvable instances where the baseline solver takes 300 seconds or less, as well as all unsat cases; (ii) focusing on problems with more than five variables ($|V| > 5$) to provide a sufficiently complex action space for the RL agent; and (iii) enforcing uniform timeouts and normalizing solver status codes. Using the processed SMTimer data, we train two predictive models (for solvability and time-bucket classification) on textual features extracted from the SMT-LIB2 formulas, with further hyperparameter details provided in the appendix.

5.2.3 Hardware and Models. All experiments were executed on a desktop workstation equipped with an Intel® Core™ i7-11700K CPU, 128 GB of RAM, and an NVIDIA GeForce RTX 3080 Ti GPU, running Ubuntu 20.04 LTS. The LLM service was hosted in a Docker container on a separate, high-performance server equipped with AMD EPYC 9654 96-Core, 1.5 TiB of RAM and NVIDIA H800 PCIe GPU. We evaluate three 70-billion-parameter LLMs: LLaMA 3.1, LLaMA 3.3, and DeepSeek-R1. All models were obtained from the official Ollama library using their respective 70B tags (e.g., ollama pull llama3.1:70b). The on-disk size for each model is approximately 43GB, which corresponds to a 4-bit quantized version (e.g., Q4_K_M).

5.3 RQ1: Effectiveness of COMPASS

5.3.1 Results on SMTimer. To verify the effectiveness of our method, we conducted experiments on the SMTimer dataset across multiple SMT solvers. Following SMT-COMP’s standard configuration⁸, we set a 1200-second timeout and consider a constraint “solved” if the solver returns sat within the time limit; timeouts are treated as unknown. We exclude constraints with baseline solving times below 300 seconds, since simpler cases can be efficiently solved without additional simplification. We also restricted our experiments to constraints with more than 5 variables, as the RL agent requires sufficient action space for meaningful strategy exploration.

To evaluate the portability of our framework, we test it on four SOTA solvers with diverse architectures: the widely-used general-purpose solvers Z3 [24] and CVC5 [8]; the arithmetic-focused solver MathSAT [23]; and BVParti [57], a specialized bit-vector solver that implements the dynamic variable-level partitioning approach of Zhao et al. [75]. For each solver, we compare its native configuration (Baseline) against the same solver augmented with our framework as a pre-processing layer (+COMPASS), without modifying any solver internals. We assess the impact of COMPASS based on four key metrics: (i) the change in the number of solved instances, (ii) the

²<https://github.com/facebookresearch/Pearl>

³<https://github.com/ollama/ollama>

⁴<https://www.gnu.org/software/coreutils/>

⁵<https://busybox.net/>

⁶<https://angr.io/>

⁷<https://smt-comp.github.io/2024/>

⁸SMT-COMP 2024 uses 1200-second timeouts: <https://smt-comp.github.io/2024/rules.html>

Table 2. Multi-Solver Performance Comparison on SMTimer Dataset. Here, "+" denotes applying COMPASS as a preprocessing layer to the baseline solver.

Solver	Total	Solved Constraints		Success Rate (%)		Avg. Time (s)	
		Baseline	+COMPASS	Baseline	+COMPASS	Baseline	+COMPASS
Z3	337	72	94	21.4	27.9	561	214
CVC5	555	113	357	20.4	64.3	1059	347
BVParti	33	2	19	6.1	57.6	694	100
MathSAT	150	29	100	19.3	66.7	757	450

preservation of baseline sat cases, (iii) the rate of regressions (i.e., baseline sat cases becoming unsolved), and (iv) the change in average solving time for successfully solved instances.

All solvers are evaluated using identical filtering criteria (a baseline solving time exceeding 300 seconds and an initial status of sat or unknown). Consequently, the total number of constraints in the final test set for each solver differs. This variation is an intentional and necessary aspect of our per-solver evaluation design, as the different totals directly reflect the varying baseline capabilities of each solver on the initial dataset, thus ensuring a fair comparison of the improvements provided by COMPASS. Table 2 summarizes the results, showing that COMPASS consistently increases the number of solved instances and reduces average solving time across all tested solvers, without requiring any internal modifications. The improvements are substantial: for Z3, the number of solved instances increases from 72 to 94 (+30.6%), with a $2.6\times$ speedup on solved cases. Similarly, for CVC5, the solved count rises dramatically from 228 to 360, a +57.9% relative improvement. While similar positive trends hold for MathSAT and BVParti, we note that the BVParti evaluation was conducted on a reduced dataset due to solver-specific bugs preventing the evaluation of numerous constraints. These results provide strong evidence that COMPASS addresses fundamental constraint solving challenges rather than exploiting solver-specific characteristics. A detailed, case-by-case analysis for each solver, including SuperVenn diagrams that illustrate the complementary nature of the solved sets, is provided in Appendix D.

5.3.2 Results on SMT-COMP. The results above confirm COMPASS's solver-agnostic benefits across diverse solvers. To facilitate the subsequent evaluations, we select Z3 as the primary backend solver. This choice is motivated by several compelling reasons. First, its widespread adoption in both academia and industry makes our findings broadly applicable. Second, its stable performance and comprehensive constraint support provide a reliable and representative baseline, unlike some specialized solvers that exhibited bugs on our dataset. Finally, its balanced baseline performance leaves substantial room for measurable improvement while maintaining statistical significance.

To evaluate scalability, we extend COMPASS to a more diverse dataset, SMT-COMP, which includes a wide variety of benchmarks. We selected the QF_LIA and QF_NIA benchmark sets from the SMT-COMP dataset. This choice is well-aligned with our focus on numerical constraints, as both QF_LIA and QF_NIA consist of quantifier-free formulas over integer variables, involving linear and nonlinear arithmetic expressions, respectively. These benchmarks are standard in evaluating SMT solvers' performance on numeric reasoning tasks and provide a diverse set of challenges that test the solver's ability to handle various arithmetic complexities. By applying our method to these benchmarks, we aim to demonstrate its effectiveness and robustness in solving a broad range of numerical constraints beyond those derived from real-world programs like BusyBox. We selected the constraints labeled as sat and unknown among these two types of constraints for solving, used 1200 seconds as the threshold, and collected the solving results. We adopt a fixed data split determined once and reused across research questions to train two prediction models (details

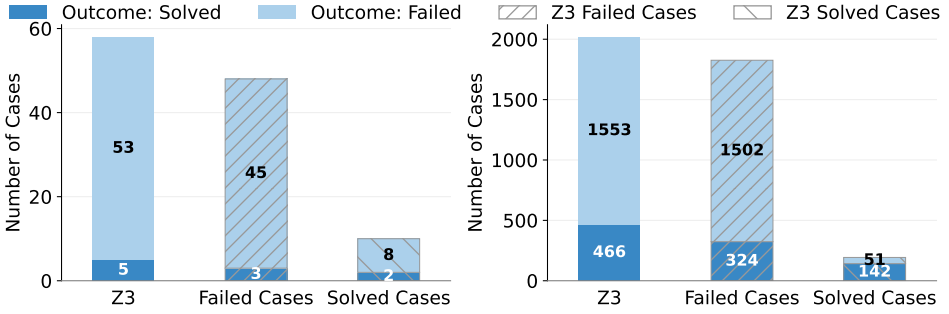


Fig. 4. Scalability on SMT-COMP (QF_LIA, QF_NIA)

in the Appendix). Among the evaluation subset, constraints with baseline solving time greater than 300 seconds and non-unsat results were taken as experimental subjects.

As shown in Figure 4, our framework scales effectively to the diverse benchmarks from the SMT-COMP dataset, demonstrating its ability to generalize beyond program-derived constraints. Compared to the Z3 baseline, COMPASS achieves a +4.6% improvement on QF_LIA and a +12.3% improvement on QF_NIA. The performance gain is particularly pronounced on the challenging nonlinear integer arithmetic (QF_NIA) problems, where COMPASS successfully converts 324 instances that previously timed out (unknown) into solved (sat) cases. Furthermore, the framework preserves 74% of the cases that the baseline Z3 could already solve, indicating that its benefits are largely complementary and not limited to simply accelerating already-solvable instances.

5.4 RQ2: Ablation Study of COMPASS

5.4.1 LLM Ablation Study.

To understand the impact of different LLMs on overall performance, we evaluate three SOTA LLMs: LLaMA 3.1:70B, LLaMA 3.3:70B, and DeepSeek-R1:70B, denoted as COMPASS_{L3.1}, COMPASS_{L3.3}, and COM-

Table 3. Performance comparison across different language models on SM-Timer dataset.

Model Variant	Solved	Success Rate (%)	Avg. Time (s)
COMPASS _{L3.1}	94	27.9	214
COMPASS _{L3.3}	89	26.4	231
COMPASS _{R1}	87	25.8	248

PASS_{R1} respectively. The results, summarized in Table 3 and visualized in Figure ??, show that the choice of LLM significantly affects performance. Table 3 provides a quantitative summary, showing that COMPASS_{L3.1} leads across all primary metrics: it achieves the highest solved count (94), the best success rate (27.9%), and the lowest average solving time (214s). This represents a substantial +30.6% improvement over the Z3 baseline (72 solved) and consistently outperforms both COMPASS_{L3.3} (89 solved) and COMPASS_{R1} (87 solved). Besides as shown in Figure ??, COMPASS_{L3.1} successfully resolves 71 of the 72 instances (a 98.6% preservation rate) that the Z3 baseline could already handle, indicating its robustness. In contrast, COMPASS_{L3.3} and COMPASS_{R1} show lower preservation rates, solving 67 and 50 of the baseline cases, respectively.

The superior performance of LLaMA 3.1:70B can be attributed to its training data, which included a significant portion of code and other formal content. This appears to have endowed it with a stronger capability for structured reasoning, allowing it to generate concise and effective

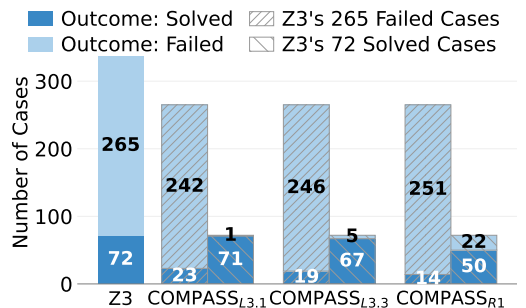


Fig. 5. LLM Model Comparison on SMTimer

values that align well with the symbolic nature of SMT formulas. In contrast, while LLaMA 3.3:70B is built upon an improved version of the 3.1 model, its fine-tuning appears to prioritize dialogue fluency over strict formal reasoning. This tendency introduced unnecessary verbosity that reduced its efficiency on this highly structured task. Similarly, DeepSeek-R1:70B, despite its recognized strengths in general and mathematical reasoning, demonstrated a tendency to reframe the problem into a more general reasoning narrative rather than adhering closely to the strict symbolic format, resulting in less effective simplification. Based on its superior performance in this specialized domain, we select COMPASS_{L3.1} as the default configuration for all subsequent experiments.

5.4.2 Component Ablation

Study. To evaluate the contributions of each component, we conducted ablation studies comparing our full RL+LLM method against four baseline configurations: (i) Random+Random (both variable and value assignment are random), (ii) LLM (LLM handles both assignments without RL guidance), (iii) Random+LLM (random variable selection with LLM value assignment), and (iv) RL+Random (RL variable selection with random values).

Table 4. Component ablation study results on SMTimer dataset (Total=337 constraints).

Method	Solved	Success Rate (%)	Avg. Time (s)
Random+Random	62	18.4	416.8
LLM	30	8.9	4.1
Random+LLM	62	18.4	334.3
RL+Random	84	24.9	429.0
RL+LLM	94	27.9	213.9

The results of this study, summarized in Table 4 and visualized in the performance profile plot in Figure 6, reveal a clear synergy between the two components. The full RL+LLM configuration significantly outperforms all baselines, achieving the highest solved count (94) and the fastest average time on solved instances (213.9s). The most critical insight comes from comparing the partial configurations: the RL+Random variant (84 solved) dramatically outperforms the Random+LLM variant (62 solved). This provides strong evidence that strategic variable selection, guided by the RL agent, is the primary driver of performance, while semantically-guided value proposal, though beneficial, is secondary. Conversely, the LLM approach performs poorly on its own (30 solved); its low average solving time (4.1s) indicates that it quickly solves trivial cases but lacks the strategic capability to handle more complex ones. The performance profile in Figure 6 further corroborates these findings by illustrating the time-to-solution dynamics. The RL+LLM curve dominates all others, as it extends furthest to the right (solving the most instances) while remaining the lowest (solving them fastest). The plot also clearly shows that both variants relying on random variable selection (Random+Random and Random+LLM) reach their limit early at 62 solved instances, reinforcing that variable selection is the key bottleneck. Conversely, the LLM curve is the shortest and flattest, visually confirming that it only solves a small number of the easiest cases very quickly before failing on more complex problems.

5.5 RQ3: Effectiveness of Selective Simplification

While RQ1 and RQ2 demonstrate COMPASS’s effectiveness and component synergy, a practical question remains regarding its real-world viability. COMPASS’s AI-driven process, while powerful, introduces computational overhead compared to direct solving. Applying this expensive preprocessing to every constraint would be impractical in production environments, as many

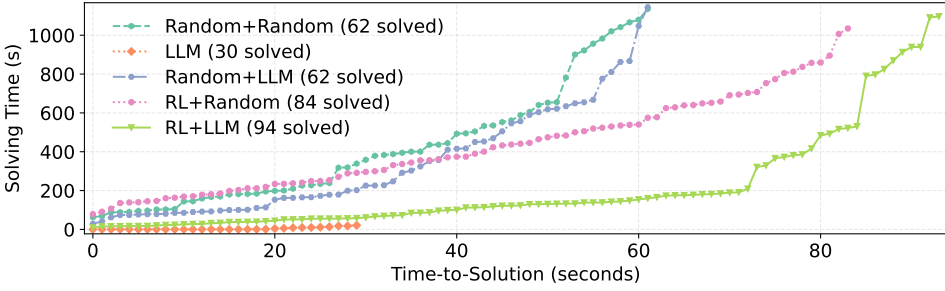


Fig. 6. The performance of COMPASS on the SMTimer with different components.

simpler problems can be solved more efficiently by the baseline solver alone. Therefore, an effective deployment strategy must selectively apply COMPASS only to those constraints identified as sufficiently complex to benefit from simplification.

To investigate such a selective simplification strategy, we simulate a production environment where the decision to apply COMPASS preprocessing is guided by predicted constraint complexity. We leverage the same QF_NIA benchmark established in our SMT-COMP evaluation, using the complete test set (10,043 constraints) with Z3 as the backend solver. The routing decision is driven by the time-bucket predictor from our pre-trained prediction models, configured with a threshold (≥ 4) that identifies approximately 0.8% of constraints as candidates for COMPASS preprocessing. We evaluate the selective simplification strategy by sending easy constraints to direct solving and hard ones to COMPASS according to the predictor. Effectiveness is measured by outcome counts and time efficiency. Table 5 presents a comprehensive comparison between direct solving and our selective simplification approach, demonstrating the time reduction benefits achieved through selective COMPASS application while maintaining solution quality across all satisfiability categories. Compared to direct solving, the selective simplification approach reduces sat average time by 12.6% (28.3s $\rightarrow$ 24.7s) and overall average by 11.9% (36.6s $\rightarrow$ 32.2s), saving 43,988s in total and converting 81 unknown cases to sat. The efficiency gains are achieved by applying COMPASS to only 0.8% of constraints, demonstrating strong performance improvements with minimal computational overhead. In practice, this selective simplification delivers tangible deployment benefits across satisfiability categories while maintaining scalability.

6 Discussion

6.1 Limitations

While COMPASS demonstrates notable solving time reductions on structured constraints, its robustness varies due to the introduction of RL and LLM. First, the effectiveness of RL, typically RL agents, heavily depends on the quality of the training set. Second, COMPASS under-performs on out-of-distribution constraints, which may cause COMPASS to consistently select incorrect variables.

Table 5. Performance comparison between direct solving and selective simplification on QF_NIA test set (10,043 constraints)

Strategy	sat Count	unsat Count	unknown Count	sat Avg. Time (s)	Overall Avg. Time (s)	Total Time (s)
Direct Solving	6,711	792	2,540	28.3	36.6	367,374
Selective Simplification	6,792	792	2,459	24.7	32.2	323,386

Third, the performance of LLM can vary. In this paper, we have selected the most effective LLM by evaluating three popular LLMs. However, it may introduce extensive effort to select the best one among all existing LLMs. Finally, COMPASS is less effective on purely linear constraints, where concretization provides minimal benefit. This limitation stems from the fundamental nature of linear constraint solving and our method's design principles. Linear constraints, by definition, involve only first-degree polynomial relationships between variables, which modern SMT solvers like Z3 handle exceptionally well through specialized algorithms such as the simplex method and Fourier-Motzkin elimination [?]. These algorithms are highly optimized for linear arithmetic and can efficiently navigate the convex solution space without requiring the strategic variable concretization that COMPASS provides.

We observed this limitation empirically during our evaluation on the SMT-COMP QF_LIA benchmark, where COMPASS achieved a modest +4.6% improvement compared to the more substantial +12.3% improvement on QF_NIA (nonlinear) constraints. The underlying reason is that linear constraints lack the complex variable coupling and nonconvex search spaces that make strategic concretization beneficial. In linear systems, variables interact through simple additive relationships, and the constraint satisfaction problem reduces to finding points within well-defined polytopes. Concretizing variables in such systems often over-constrains the problem without providing the simplification benefits observed in nonlinear cases, where variable interactions involve products, powers, and modular arithmetic that create exponential search complexity.

6.2 Threats to Validity

Several potential threats to the validity of our findings should be considered when interpreting the experimental results.

Internal Validity. Our primary results are based on a specific configuration using LLaMA 3.1:70B as the LLM backend and Soft Actor-Critic (SAC) as the RL algorithm. While we conducted comparative evaluations with alternative LLM models (RQ2) and component ablations to validate design choices, the headline performance metrics reflect this primary configuration. The stochastic nature of both LLM generation and RL exploration introduces inherent variability in results. To mitigate these concerns, we employed several strategies: (i) consistent preprocessing pipelines across all experiments, (ii) solver-in-the-loop verification of every LLM-suggested assignment to ensure soundness, (iii) systematic ablation studies (RQ2) to isolate component contributions, and (iv) cross-solver validation (RQ1) to demonstrate architecture-agnostic benefits. All experiments were conducted under controlled computational environments with uniform timeout settings (1200 seconds, following SMT-COMP standards) to ensure reproducibility.

External Validity. Our evaluation encompasses two distinct benchmark categories with different theoretical foundations. The SMTimer dataset consists primarily of program-derived constraints involving bit-vector arithmetic from real-world software (Coreutils, BusyBox), while the SMT-COMP benchmarks provide QF_LIA and QF_NIA constraints focusing on linear and nonlinear integer arithmetic. This dual-dataset approach, while providing broader coverage than single-theory evaluations, still represents a limited subset of the full SMT theory spectrum. Key domains such as quantified formulas, arrays, strings, floating-point arithmetic, and uninterpreted functions remain largely unexplored beyond our specialized BVParti evaluation on bit-vector constraints.

Furthermore, our implementation relies on established open-source frameworks (Pearl for reinforcement learning and Ollama for LLM hosting), which introduces potential dependency-related validity concerns. While these frameworks provide robust, well-tested foundations that enhance reproducibility, they also constrain our experimental flexibility and may introduce framework-specific behaviors that could affect generalizability. The Pearl framework's SAC implementation

and Ollama’s model quantization strategies (Q4_K_M format) represent specific algorithmic choices that may not generalize to alternative RL algorithms or LLM deployment methods.

Additionally, our constraint filtering criteria (baseline solving time >300 seconds, >5 variables, non-unsat cases) may introduce selection bias toward specific problem characteristics, particularly favoring constraints with sufficient complexity to benefit from our approach while potentially excluding simpler cases where direct solving is more appropriate. The generalizability of our approach to other SMT theories, alternative constraint distributions, and different implementation frameworks requires further investigation.

Construct Validity. Our evaluation metrics—solving time and success rate—are inherently solver-centric and may not fully capture the complexity of constraint simplification effectiveness. These metrics are closely tied to the internal decision procedures and heuristics of specific solvers, potentially limiting the generalizability of performance comparisons. While we validated trends across multiple solver architectures (Z3, CVC5, MathSAT, BVPARTI) to address this concern, each solver’s unique approach to handling concretized constraints may influence results differently. Furthermore, our hybrid reward system combines multiple signal sources (solver outcomes, predictive models, history-based penalties), and the relative weighting of these components may affect learning dynamics in ways not fully captured by end-to-end performance metrics.

Conclusion Validity. The choice of evaluation parameters can significantly influence result interpretation. Our adoption of the standard 1200-second timeout threshold, while consistent with SMT-COMP practices, may particularly impact constraints near this boundary where small performance shifts can change outcome classifications. Additionally, our selective simplification evaluation (RQ3) uses a specific routing threshold (time-bucket 4) that identifies approximately 0.8% of constraints for COMPASS processing. This threshold was chosen for demonstration rather than optimization, and alternative thresholds may yield different cost-benefit trade-offs. The statistical significance of improvements, particularly for smaller datasets like BVPARTI (33 constraints), may be limited by sample size constraints.

These validity considerations underscore the importance of diverse experimental design, comprehensive cross-solver validation, and rigorous benchmarking when integrating machine learning approaches into symbolic reasoning systems. Future work should address these limitations through broader theory coverage, alternative evaluation metrics, and systematic sensitivity analyses across key experimental parameters.

7 Related Work

The performance of SMT solvers is a critical bottleneck in areas like symbolic execution [16, 56, 68], especially when faced with complex, non-linear constraints [14]. Prior work has tackled this challenge from multiple angles. We organize the literature into two primary layers—solver-level enhancement and constraint-level simplification—and position COMPASS as a policy-learning paradigm for *pre-solver* constraint simplification via variable-level semantic concretization.

7.1 Solver-Level Enhancement

Solver enhancement approaches modify SMT solvers directly. Widely used SMT backends such as STP [26], Z3 [24], CVC5 [8], MathSAT5 [23], Boolector [45], and BVPARTI [57] provide a foundation for solver development. Algorithm selection methods [14, 54] use ML to optimize solver configurations and predict solving times; online portfolio selection such as MedleySolver [49] further schedules backends adaptively. Internal modifications include neural branching heuristics [34, 63] and distributed architectures [75] that partition constraints across multiple instances. This solver-level line of work reduces the effective search space via partitioning; by contrast, COMPASS reduces search space *pre-solver* through variable-level semantic concretization without touching

solver internals. Beyond this, theory arbitrage transforms unbounded constraints into bounded theories to accelerate solving [41]. Neural solvers [1, 55] attempt complete replacement but struggle with complex problems. These approaches are orthogonal to COMPASS—they build better solvers, whereas COMPASS reduces difficulty pre-solver without modifying backends and can be composed with them.

In parallel, LLM–solver integration has emerged within this axis, such as Logic-LM [48] for integrating LLMs with logical solvers and MCP-Solver [61] for constraint programming. These methods strengthen backends or interfaces but, unlike COMPASS, do not perform variable-level, semantics-aware concretization prior to solving.

7.2 Constraint-Level Simplification

Constraint simplification approaches use AI to transform constraints before solving. Deep learning methods [64, 67] apply neural networks for constraint prediction and ML-guided symbolic analysis. Strategy synthesis approaches [20] automatically generate solving strategies for symbolic execution, while time prediction methods [39, 62] use ML to predict constraint solving complexity. Rewrite-based simplification via equality saturation (e.g., egg [65]) emphasizes canonicalization rather than goal-directed, variable-level concretization. Specialized approaches target specific constraint types [37, 58], while hybrid methods [42] combine SMT solving with fuzzing techniques. Recent work learns patterns to bias symbolic execution toward vulnerable states [53], and further converts SMT formulas to effective test cases to boost testing pipelines [73]. The latest developments include concrete constraint guidance for symbolic execution [4] and specialized vulnerability signature generation using static analysis guidance [74]; COMPASS instead reduces solver difficulty via variable-level, semantics-aware concretization and composes with these efforts. Unlike equality saturation or handcrafted rewrite rules, COMPASS performs goal-directed, per-variable concretization guided by learned, semantics-aware policies.

At the constraint level, LLM-based methods have been considered for adjacent tasks such as translating natural language to logical formulas (SatLM [69]); however, they generally do not learn variable-level, semantics-aware concretization policies for SMT formulas, which is the focus of COMPASS. Taken together, existing work optimizes solvers or applies fixed rewrite strategies; COMPASS instead offers a cross-cutting, pre-solver strategy that learns semantics-aware, variable-level concretization to reduce solver difficulty. COMPASS operates on a fundamentally different principle from existing constraint simplification approaches. Unlike Wen et al. [64] who use deep learning for general constraint solving, or Yang et al. [67] who apply ML prediction for symbolic analysis guidance, COMPASS combines RL and LLM for strategic variable concretization and learns a fine-grained, variable-level policy that actively reduces problem complexity through semantics-aware reasoning. This synergy—where the RL agent provides long-term strategy and the LLM offers semantic intuition for each step—distinguishes COMPASS from approaches that focus on high-level tactic selection or time prediction [19, 38, 39] and from LLM-based interfaces that do not provide learned, variable-level, semantics-aware concretization for SMT formulas. Overall, LLMs have been applied to formal reasoning and solver integration [19, 30, 48, 61, 69]. These techniques largely follow guess-and-verify paradigms with scalability limitations and are complementary to COMPASS.

8 Conclusion

This work presented COMPASS, a solver-agnostic framework that casts SMT constraint simplification as policy learning: an RL agent selects variables, an LLM proposes values, and the solver verifies soundness in the loop. Across heterogeneous backends, COMPASS consistently increases solved counts and reduces time (e.g., Z3: 72→94, +30.6%; CVC5: 228→360, +57.9%; BVParti: 2→19,

+850%; MathSAT: 29→100, +244.8%; see Section 5). Ablations confirm the synergy between policy guidance and semantic value proposals, and cross-solver results (RQ4) indicate generalizability rather than solver-specific overfitting.

Looking ahead toward practical neuro-symbolic co-pilots [31], we will: (i) expand theory coverage to bit-vectors, arrays, and quantified theories with solver-invariant abstractions; (ii) tighten LLM–solver integration via proof objects, unsoundness rejection, and expert feedback; (iii) improve deployment with calibrated, budget/risk-aware routing (RQ5) and policy distillation/caching to cut latency; and (iv) broaden datasets and standardized benchmarks to stress-test robustness and transfer.

References

- [1] Marcelo Abreu, Rafael Giusti, Marcos Maximo, and Mariá CV Nascimento. 2021. Learning to solve NP-complete problems: A graph neural network for decision TSP. In *Advances in Neural Information Processing Systems*, Vol. 34. 21054–21066.
- [2] Uri Alon, Meital Zilberstein, Omer Levy, and Eran Yahav. 2019. code2vec: Learning Distributed Representations of Code. In *Proceedings of the ACM on Programming Languages, POPL 2019, Lisbon, Portugal, January 13-19, 2019*, Vol. 3. 40:1–40:29. <https://doi.org/10.1145/3290353>
- [3] Cornelius Aschermann, Sergej Schumilo, Robert Gawlik, Timon Danisevskis, and Thorsten Holz. 2020. RedQueen: Fuzzing with Input-to-State Correspondence. In *Network and Distributed System Security Symposium (NDSS)*.
- [4] Various Authors. 2024. Concrete Constraint Guided Symbolic Execution. In *Proceedings of the 46th International Conference on Software Engineering*. ACM.
- [5] Thanassis Avgerinos, Sang Kil Cha, Kasper Luckow, and David Brumley. 2012. Automatic Exploit Generation for Software. In *IEEE Symposium on Security and Privacy*. IEEE, 741–755.
- [6] Thanassis Avgerinos, Alexandre Rebert, Sang Kil Cha, and David Brumley. 2014. Enhancing Symbolic Execution with Veritesting. In *Proceedings of the 36th International Conference on Software Engineering*. ACM, 1083–1094.
- [7] Thomas Ball and Sriram K Rajamani. 2004. SLAM and static driver verifier: Technology transfer of formal methods inside Microsoft. In *International Conference on Integrated Formal Methods*. Springer, 1–20.
- [8] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, et al. 2022. cvc5: A versatile and industrial-strength SMT solver. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 415–442.
- [9] Clark Barrett, Aaron Stump, and Cesare Tinelli. 2010. The SMT-LIB Standard – Version 2.0. In *Proceedings of the 8th International Workshop on Satisfiability Modulo Theories (SMT '10)*. <http://theory.stanford.edu/~barrett/pubs/BST10.pdf> Edinburgh, Scotland.
- [10] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. 2018. Relational inductive biases, deep learning, and graph networks. *arXiv preprint arXiv:1806.01261* (2018).
- [11] Yoshua Bengio. 2009. Learning deep architectures for AI. *Foundations and trends® in Machine Learning* 2, 1 (2009), 1–127.
- [12] Yoshua Bengio, Aaron Courville, and Pascal Vincent. 2013. Representation learning: A review and new perspectives. *IEEE transactions on pattern analysis and machine intelligence* 35, 8 (2013), 1798–1828.
- [13] Saahil Bessler, Sunjay Cha, Michael Hicks, and Jeffrey S Foster. 2021. Metrinome: Path complexity predicts symbolic execution path explosion. In *Proceedings of the 43rd International Conference on Software Engineering*. 1134–1145.
- [14] Haniel Bischoff, Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mathias Peter, Aina Preiner, et al. 2021. Satisfiability modulo theories competition (SMT-COMP): 2020. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 374–380.
- [15] Lei Bu, Jiaxiang Zhao, Sen Li, and Jun Sun. 2021. Machine learning steered symbolic execution framework for complex software code. In *Proceedings of the 43rd International Conference on Software Engineering*. 1112–1123.
- [16] Cristian Cadar, Daniel Dunbar, Dawson R Engler, et al. 2008. KLEE: Unassisted and automatic generation of high-coverage tests for complex systems programs. In *OSDI*, Vol. 8. 209–224.
- [17] Cristian Cadar and Koushik Sen. 2013. Symbolic Execution for Software Testing: Three Decades Later. *Commun. ACM* 56, 2 (2013), 82–90.
- [18] Sooyoung Cha, Myungho Lee, Seokhyun Lee, and Hakjoo Oh. 2022. SymTuner: Maximizing the Power of Symbolic Execution by Adaptively Tuning External Parameters. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, 2068–2079. <https://doi.org/10.1145/3510003.3510185>

- [19] Supratik Chakraborty, Dror Fried, Lucas M Tabajara, and Moshe Y Vardi. 2023. Ranking llm-generated loop invariants for program verification. In *arXiv preprint arXiv:2310.09342*.
- [20] Haoyu Chen, Yinxing Xue, Yuekang Li, Sen Chen, Bihuan Xu, Yang Feng, and Lei Ma. 2021. Synthesize solving strategy for symbolic execution. In *Proceedings of the 43rd International Conference on Software Engineering*. 1022–1034.
- [21] Peng Chen, Jianzhong Liu, and Hao Chen. 2023. Baton: symphony of random testing and concolic testing. In *Proceedings of the 45th International Conference on Software Engineering*. 1567–1578.
- [22] Vitaly Chipounov, Volodymyr Kuznetsov, and George Candea. 2011. S2E: A Platform for In-vivo Multi-path Analysis of Software Systems. In *Proceedings of the Sixteenth International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 265–278.
- [23] Alessandro Cimatti, Alberto Griggio, Bastiaan Schaafsma, and Roberto Sebastiani. 2013. MathSAT5: An SMT Solver for Formal Verification. In *International Conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 93–107.
- [24] Leonardo De Moura and Nikolaj Bjørner. 2008. Z3: An efficient SMT solver. In *International conference on Tools and Algorithms for the Construction and Analysis of Systems*. Springer, 337–340.
- [25] Zhangyin Feng, Daya Guo, Duyu Tang, Nan Duan, Xiaocheng Feng, Ming Gong, Lin Shou, Bing Qin, Ting Liu, Daxin Jiang, and Ming Zhou. 2020. CodeBERT: A Pre-Trained Model for Programming and Natural Languages. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. Association for Computational Linguistics, 1536–1547. <https://www.aclweb.org/anthology/2020.findings-emnlp.139>
- [26] Vijay Ganesh and David L. Dill. 2007. STP: A Decision Procedure for Bit-Vectors and Arrays. In *Computer Aided Verification*. Springer, 238–249.
- [27] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor. In *Proceedings of the 35th International Conference on Machine Learning (ICML) (Proceedings of Machine Learning Research, Vol. 80)*, Jennifer Dy and Andreas Krause (Eds.). PMLR, 1861–1870.
- [28] Jingxuan He, Mislav Balunovic, Gagandeep Kuchta, and Martin Vechev. 2021. Learning to Explore Paths for Symbolic Execution. In *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*. 1279–1294.
- [29] Johannes Henkel, Shuo-Hsiu Hsu, Jiacheng Liu, Zsolt Zombori, Awesome S. Babalola, Timon Gehr, Georgel Călin, Rishabh Singh, and Martin Vechev. 2023. Neural Program Generation for SMT Solving. In *The Eleventh International Conference on Learning Representations, ICLR 2023, Kigali, Rwanda, May 1-5, 2023*. OpenReview.net. <https://openreview.net/forum?id=j6sVd4MIi0>
- [30] Adharsh Kamath, Aditya Senthilnathan, Supratik Chakraborty, Kuldeep S Meel, and Moshe Y Vardi. 2023. Finding inductive loop invariants using large language models. In *arXiv preprint arXiv:2311.07948*.
- [31] Henry Kautz. 2021. Neuro-symbolic artificial intelligence: The state of the art. *AI Magazine* 42, 4 (2021), 3–12.
- [32] Henry Kautz. 2022. The third AI summer: AAAI Robert S. Englemore memorial award lecture. *AI Magazine* 43, 1 (2022), 93–104.
- [33] James Kukucka, Luís Pina, Paul Ammann, and Jonathan Bell. 2022. CONFETTI: Amplifying Concolic Guidance for Fuzzers. In *Proceedings of the 44th International Conference on Software Engineering*. ACM, 1834–1846. <https://doi.org/10.1145/3510003.3510628>
- [34] Vitaly Kurin, Saad Godil, Shimon Whiteson, and Bryan Catanzaro. 2020. Improving SAT solver heuristics with graph networks and reinforcement learning. In *arXiv preprint arXiv:2001.07105*.
- [35] Michael Laskin, Aravind Rajeswaran, Tim Rocktäschel, Misha Denil, and Pieter Abbeel. 2021. URLB: Unsupervised Reinforcement Learning Benchmark. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 18838–18850. <https://proceedings.neurips.cc/paper/2021/file/99e226084439203932e673e5667e41e3-Paper.pdf>
- [36] Sergey Levine, Aviral Kumar, George Tucker, and Justin Fu. 2020. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. *arXiv preprint arXiv:2005.01643* (2020).
- [37] Meixi Liu, Ziqi Shuai, Luyao Liu, Keli Ma, and Ke Ma. 2022. Optimal Refinement-based Array Constraint Solving for Symbolic Execution. In *2022 29th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 299–308. <https://doi.org/10.1109/APSEC57359.2022.00042>
- [38] Haiming Luo, Haoxiong Chang, Qingxing Huang, Jing Shi, Yinya Zhang, and Lei Li. 2023. Learning to prove theorems by learning to generate theorems. In *International Conference on Machine Learning*. PMLR, 22278–22300.
- [39] Sicheng Luo, Hui Xu, Yanxiang Bi, Xin Wang, and Yangfan Zhou. 2021. Boosting Symbolic Execution via Constraint Solving Time Prediction (Experience Paper). In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 336–347. <https://doi.org/10.1145/3460319.3464813>
- [40] Sergey Mechtaev, Xiang Gao, Abhik Roychoudhury, and Satish Chandra. 2018. Semantic program repair using a reference implementation. In *Proceedings of the 40th International Conference on Software Engineering*. 129–139.

- [41] Benjamin Mikek and Qirun Zhang. 2024. SMT Theory Arbitrage: Approximating Unbounded Constraints using Bounded Theories. In *Proceedings of the 45th ACM SIGPLAN Conference on Programming Language Design and Implementation*. ACM.
- [42] Sujit Kumar Muduli and Subhajit Roy. 2022. Satisfiability modulo Fuzzing: A Synergistic Combination of SMT Solving and Fuzzing. *Proceedings of the ACM on Programming Languages* 6, OOPSLA2 (2022), 169:1236–169:1263. <https://doi.org/10.1145/3563332>
- [43] Authors Multiple. 2024. Logos: Log Guided Fuzzing for Protocol Implementations. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM.
- [44] Katta G. Murty and Santosh N. Kabadi. 2002. The Complexity of Nonlinear Integer Programming. *Discrete Applied Mathematics* 123, 1-3 (2002), 75–103.
- [45] Aina Niemetz, Mathias Preiner, Armin Biere, and Martin Zisser. 2018. Boolector 3.0. In *Computer Aided Verification*. Springer, 227–245.
- [46] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. *Advances in neural information processing systems* 35 (2022), 27730–27744.
- [47] Ciprian Paduraru, Mihai-Cristian Melemtciuc, and Alin Stefanescu. 2020. Optimizing decision making in concolic execution using reinforcement learning. In *Proceedings of the 2020 Genetic and Evolutionary Computation Conference*. 1423–1431.
- [48] Liangming Pan, Alon Albalak, Xinyi Wang, and William Yang Wang. 2023. Logic-LM: Empowering Large Language Models with Symbolic Solvers for Faithful Logical Reasoning. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 3806–3824.
- [49] Nikhil Pimpalkhare, Federico Mora, Elizabeth Polgreen, and Sanjit A. Seshia. 2021. MedleySolver: Online SMT Algorithm Selection. In *Theory and Applications of Satisfiability Testing – SAT 2021*. Springer, 558–576. https://doi.org/10.1007/978-3-030-80223-3_31
- [50] Sebastian Poeplau and Aurélien Francillon. 2020. Symbolic Execution with SymCC: Don’t Interpret, Compile!. In *29th USENIX Security Symposium (USENIX Security 20)*. 181–198.
- [51] Sebastian Poeplau and Aurélien Francillon. 2021. SymQEMU: Compilation-based Symbolic Execution for Binaries. In *Proceedings 2021 Network and Distributed System Security Symposium*. Internet Society. <https://doi.org/10.14722/ndss.2021.24118>
- [52] Sanjay Rawat, Vivek Jain, Ashish Kumar, Lucian Cojocar, Cristiano Giuffrida, and Herbert Bos. 2017. VUzzer: Application-aware evolutionary fuzzing. In *Proceedings of the 2017 Network and Distributed System Security Symposium*.
- [53] Vincenzo Ruaro, Giuseppe Antonio Di Luna, Carlo Della Corte, Bjorn De Sutter, Giovanni Vigna, and Andrea Continella. 2021. SyML: Guiding Symbolic Execution Toward Vulnerable States Through Pattern Learning. In *Proceedings of the 24th International Symposium on Research in Attacks, Intrusions and Defenses*. ACM, 559–574.
- [54] Joseph Scott, Aina Niemetz, Mathias Preiner, Saeed Nejati, and Vijay Ganesh. 2023. Algorithm selection for SMT. In *Proceedings of the 2023 ACM SIGPLAN International Symposium on Memory Management*. 1–15.
- [55] Daniel Selsam, Matthew Lamm, Benedikt Bunz, Percy Liang, Leonardo de Moura, and David L Dill. 2019. Learning a SAT solver from single-bit supervision. In *International Conference on Learning Representations*.
- [56] Koushik Sen, Darko Marinov, and Gul Agha. 2005. CUTE: a concolic unit testing engine for C. In *ACM SIGSOFT Software Engineering Notes*, Vol. 30. ACM, 263–272.
- [57] Shaowei Cai Group. 2025. STP-Parti-Bitwuzla at SMT-COMP 2025 (BVParti). GitHub release. <https://github.com/shaowei-cai-group/STP-Parti-Bitwuzla-at-SMT-COMP-2025/releases/tag/v1.0.0> Bit-vector implementation of dynamic variable-level partitioning by Zhao et al. (CAV 2024) [75].
- [58] Ziqi Shuai, Zhenbang Chen, Yufeng Zhang, Jun Sun, and Ji Wang. 2021. Type and Interval Aware Array Constraint Solving for Symbolic Execution. In *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*. ACM, 361–373. <https://doi.org/10.1145/3460319.3464826>
- [59] Nick Stephens, John Grosen, Christopher Salls, Andrew Dutcher, Ruoyu Wang, Jacopo Corbetta, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. 2016. Driller: Augmenting fuzzing through selective symbolic execution. In *25th USENIX Security Symposium*. 1051–1066.
- [60] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (second ed.). The MIT Press.
- [61] Stefan Szeider. 2025. MCP-Solver: Integrating Language Models with Constraint Programming Systems. *arXiv preprint arXiv:2501.00539* (2025).
- [62] Nicola Thoben, Jan Haltermann, and Heike Wehrheim. 2023. Timeout Prediction for Software Analyses. In *Software Engineering and Formal Methods*, Carla Ferreira and Tim A. C. Willemse (Eds.), Vol. 14323. Springer Nature Switzerland, 340–358. https://doi.org/10.1007/978-3-031-47115-5_19
- [63] Wenxi Wang, Yangzihao Yang, Qingyu Li, Yang Wang, Chenyang Liu, and Holger H Hoos. 2021. NeuroComb: Improving SAT solving with graph neural networks. In *arXiv preprint arXiv:2110.14053*.

- [64] Junye Wen, Mujahid Khan, Meiru Che, Yan Yan, and Guowei Yang. 2020. Constraint Solving with Deep Learning for Symbolic Execution. <https://doi.org/10.48550/arXiv.2003.08350> arXiv:2003.08350 [cs.SE]
- [65] Max Willsey, Yisu Remy Wang, Philip Nandi, Matthew Flatt, Zachary Tatlock, and Emina Torlak. 2021. egg: Fast and Extensible Equality Saturation. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*. ACM, 335–349.
- [66] Junjie Wu, Zhiqiang Tang, Qingkai Zeng, and Peng Wang. 2020. Reinforcement Learning Guided Symbolic Execution. In *Proceedings of the 35th IEEE/ACM International Conference on Automated Software Engineering*. 1129–1141.
- [67] Mingyue Yang, David Lie, and Nicolas Papernot. 2024. Exploring Strategies for Guiding Symbolic Analysis with Machine Learning Prediction. In *2024 IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*. IEEE, 659–669. <https://doi.org/10.1109/SANER60148.2024.00073>
- [68] Xuejun Yang, Yang Chen, Eric Eide, and John Regehr. 2011. CSmith: a random generator of C programs. In *ACM SIGPLAN Notices*, Vol. 46. ACM, 430–440.
- [69] Xi Ye, Qiaochu Chen, Isil Dillig, and Greg Durrett. 2023. SatLM: Satisfiability-Aided Language Models Using Declarative Prompting. In *Advances in Neural Information Processing Systems*, Vol. 36. Curran Associates, Inc., 20393–20411.
- [70] Jaehan Yoon and Sooyoung Cha. 2024. FeatMaker: Automated Feature Engineering for Search Strategy of Symbolic Execution. In *Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering*. ACM, 1–13. <https://doi.org/10.1145/3660815>
- [71] Insu Yun, Sangho Lee, Meng Xu, Yeongjin Jang, and Taesoo Kim. 2018. QSYM: A Practical Concolic Execution Engine Tailored for Hybrid Fuzzing. In *27th USENIX Security Symposium (USENIX Security 18)*. 745–761.
- [72] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. 2017. Deep Sets. In *Advances in Neural Information Processing Systems*, Vol. 30. 3391–3401.
- [73] Chengyu Zhang and Zhendong Su. 2024. SMT2Test: From SMT Formulas to Effective Test Cases. In *Proceedings of the ACM on Programming Languages (OOPSLA)*. ACM. <https://doi.org/10.1145/3689719>
- [74] Yiwei Zhang, Heng Wang, Yuekang Li, Yang Liu, et al. 2024. STASE: Static Analysis Guided Symbolic Execution for UEFI Vulnerability Signature Generation. In *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. IEEE/ACM.
- [75] Mengyu Zhao, Shaowei Cai, and Yuhang Qian. 2024. Distributed SMT Solving Based on Dynamic Variable-Level Partitioning. In *Computer Aided Verification: 36th International Conference, CAV 2024 (Lecture Notes in Computer Science, Vol. 14681)*. Springer, 67–89. https://doi.org/10.1007/978-3-031-65627-9_4
- [76] Shunfan Zhou, Zheming Yang, Dan Qiao, Peng Liu, Min Yang, Zhe Wang, and Chenggang Wu. 2022. Ferry: State-Aware Symbolic Execution for Exploring State-Dependent Program Paths. In *31st USENIX Security Symposium (USENIX Security 22)*. USENIX Association, 4365–4382. <https://www.usenix.org/conference/usenixsecurity22/presentation/zhou-shunfan>

A Dataset Overview and Filtering Pipeline

Table 6. Overview of the SMTimer dataset and baseline Z3 results.

Tool	SAT	UNSAT	UNKNOWN	SAT Time (s)	UNSAT Time (s)	UNKNOWN Time (s)
busybox	3,802	13,359	155	7,455.48	1,496,955.43	186,000
gnu	32,495	37,529	520	409,265.42	369,984.23	624,000

Data Filtering and Preprocessing Pipeline. Before conducting experiments, we apply a rigorous filtering pipeline to ensure fair comparison and meaningful results:

- (1) **Initial solver filtering:** We use multiple SMT solvers to filter the original dataset, removing trivial or unsolvable constraints to focus on meaningful test cases.
- (2) **Baseline time filtering:** Constraints that can be solved by baseline solvers in ≤ 300 seconds are excluded, focusing evaluation on challenging instances where our method can demonstrate advantages.
- (3) **Variable count threshold:** Based on our experimental analysis, we restrict evaluation to constraints with $|V| > 5$ variables, where V is the set of symbolic variables after normalization. This threshold ensures the RL agent has sufficient action space for meaningful strategy exploration.

The filtering process results in different numbers of constraints for each solver due to their varying solving capabilities and performance characteristics. The exact number of filtered constraints varies depending on the specific solver being evaluated and will be reported in the corresponding experimental results.

B Predictive Models and Implementation Details

Terminology and Discretization. We use solver outcomes sat/unsat/unknown throughout. We discretize wall-clock solving time on $[0, 1000]$ seconds into 8 equal-width *time buckets (intervals)*; unless otherwise noted, “bucket” and “interval” are used interchangeably. Given a batch size B , model inputs are matrices $X \in \mathbb{R}^{B \times d}$ with feature dimension d specified below.

Encoders. SMTimer (CodeBERT): SMT-LIB scripts are serialized and encoded with a pre-trained CodeBERT, producing 768-dimensional embeddings per instance (see Section 4). These representations are consumed by both the RL agent and the predictors; a mini-batch forms $X \in \mathbb{R}^{B \times 768}$.

SMT-COMP (LLM-based): For large SMT-COMP inputs, fixed-size embeddings are obtained from a deployed LLM-based embedding service (approximately 70B parameters). Each instance yields an 8192-dimensional vector; a mini-batch forms $X \in \mathbb{R}^{B \times 8192}$.

Predictors (Unified Presentation). We use two predictors across both datasets; only the input dimension d differs by encoder.

Binary Solvability Predictor $P_{\text{solve}} : \mathbb{R}^d \rightarrow [0, 1]$. - *Input:* 768-d (SMTimer) or 8192-d (SMT-COMP). - *Architecture:*

- SMTimer: Two-layer MLP ($768 \rightarrow 128 \rightarrow 1$) with ReLU activation.
- SMT-COMP: Multi-layer feed-forward network with residual connections operating over 8192-dimensional features.

- *Loss/Target:* BCE-with-logits with target $y \in \{0, 1\}$, where $y = 1$ denotes sat and $y = 0$ denotes non-sat. - *Training:* An 80/20 train/validation split, batch size 64, Adam (1e-3), ReduceLROnPlateau, and early stopping (patience 10) are adopted; the same predictor is reused per dataset and solver across RQ1–RQ5.

Multi-class Time Predictor $P_{\text{time}} : \mathbb{R}^d \rightarrow \mathbb{R}^8$. - *Input:* 768-d (SMTimer) or 8192-d (SMT-COMP). - *Architecture:*

- SMTimer: Four-layer MLP ($768 \rightarrow 256 \rightarrow 128 \rightarrow 64 \rightarrow 8$) with batch normalization, dropout (0.5), Leaky ReLU, and a residual connection ($768 \rightarrow 8$).
- SMT-COMP: Deep MLP with batch normalization, dropout (0.5), Leaky ReLU, and a residual linear head mapping 8192 to 8.

- *Loss/Target:* Cross-entropy over 8 time buckets (intervals) spanning $[0, 1000]$ seconds. - *Training:* An 80/20 train/validation split, batch size 64; SMTimer uses Adam (1e-3) with 400 epochs, SMT-COMP uses Adam (1e-3) with weight decay (1e-4); both employ ReduceLROnPlateau and early stopping.

Training Protocol and Data Splits. Across RQ1–RQ5, half of the data are used for training and the remainder for evaluation, focusing on non-unsat cases and baseline time > 300 seconds for fairness. For RQ5, settings follow this appendix; the routing threshold is based on predicted time bucket > 4 .

Integration with RL and Timeouts. Predictors supply dense guidance during exploration and inform dynamic timeout selection. A global 1200-second episode timeout is used, with per-step timeouts derived from P_{time} to avoid excessive time on instances likely to occupy higher buckets while still allocating sufficient budget to sat cases in mid/low buckets.

System and Model Components. Our implementation builds on Pearl⁹ and Ollama¹⁰. The RL agent uses SAC. The LLM component uses LLaMA 3.1:70B for value generation and heuristic guidance. Unless otherwise noted, other low-level configurations (tokenization, batch sizes, optimization) follow Section 4.

C Motivating Example: Variable Impact Scores

This section provides the detailed per-variable Impact Scores (Frequency F, Complexity C, Coupling P; total $I = F + C + P$) referenced in Section 3.1. Variable z is the only one that appears in all four constraints and has the highest total score ($I = 11$), matching the analysis in the main text.

Table 7. Per-variable impact scores for the motivating example. Higher I suggests higher priority for concretization.

Variable	F	C	P	$I = F + C + P$
x	4	2	2	8
y	3	2	2	7
z	4	3	4	11
a	3	2	3	8
b	2	1	3	6
c	2	0	2	4

D Detailed SuperVenn Analysis and Case Studies

SuperVenn Analysis and Detailed Case Studies To provide deeper insights into how the COMPASS method enhances different solvers, this section employs SuperVenn diagrams to visualize the solving capability relationships. The diagrams reveal not only the quantitative improvements but also the complementary nature of solvers and their COMPASS-enhanced versions.

Z3 Enhancement Analysis Figure 7 presents a SuperVenn analysis of Z3’s enhancement, showing that the method solved 94 constraints compared to Z3’s baseline 72 constraints (30.6% improvement). The analysis reveals that the method solved 23 additional constraints that the Z3 baseline could not handle, while maintaining the ability to solve 71 of the 72 constraints that Z3 originally solved (98.6% preservation rate).

One representative constraint from the 23 Z3+COMPASS-exclusive cases was examined in detail. This constraint involves 47 integer variables with complex arithmetic relationships including modular operations, conditional expressions, and nested quantifications. The constraint defines multiple helper functions for range checking and arithmetic validation, with assertions that impose intricate dependencies between variables through chains of implications and equivalences. The overall objective is to determine satisfiability of a system where variables must satisfy simultaneous linear and nonlinear arithmetic constraints with strict bounds checking.

Z3 baseline failed on this constraint due to the combinatorial explosion in the search space—with 47 variables each having large integer domains, the traditional symbolic approach struggled to efficiently explore the constraint space within the timeout. The COMPASS approach succeeded by strategically concretizing 19 key variables (identified through RL guidance) in 3 candidate sequences over 892 seconds. The final sequence enabled Z3 to solve the simplified constraint in just 8 seconds, demonstrating that guided concretization can transform intractable problems into manageable ones.

⁹<https://github.com/facebookresearch/Pearl>

¹⁰<https://github.com/ollama/ollama>

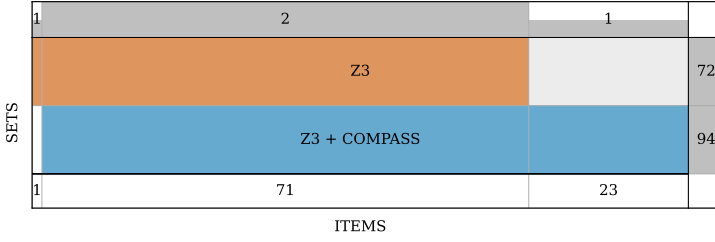


Fig. 7. SuperVenn analysis of Z3 vs Z3+COMPASS. The diagram shows 1 constraints solved exclusively by Z3, 23 constraints solved exclusively by Z3+COMPASS, and 71 constraints solved by both. Total improvement: 72 → 94 (+22 constraints, 30.6% relative).

CVC5 Enhancement Analysis Figure 8 shows an even more dramatic enhancement for CVC5, with our COMPASS method solving 360 constraints compared to CVC5's baseline 228 constraints (57.9% improvement). The method solved 317 constraints that CVC5 baseline could not handle, while 43 constraints were solved by both methods and 185 were solved exclusively by baseline CVC5.

We analyzed a representative constraint from the 317 CVC5+COMPASS-exclusive cases. This constraint defines 38 real-valued variables with complex polynomial relationships, including quadratic terms, trigonometric functions, and transcendental equations. The constraint system involves simultaneous nonlinear equations with strict inequality bounds, where variables represent geometric coordinates that must satisfy both algebraic constraints (polynomial equalities) and geometric constraints (distance and angle relationships). The assertions include nested conditional statements that create multiple solution branches, each requiring different variable assignment strategies.

CVC5 baseline struggled with this constraint because its arithmetic decision procedures, while powerful for linear arithmetic, face exponential complexity when handling the combination of nonlinear terms and branching conditions. The constraint's 38 real variables create a continuous search space that traditional symbolic methods cannot efficiently explore. Our COMPASS approach succeeded by identifying 14 critical variables through RL guidance and generating 4 candidate assignment sequences over 1,247 seconds. The successful sequence concretized these key variables, reducing the problem to a linear system that CVC5 solved in 23 seconds. This demonstrates how strategic variable concretization can transform nonlinear problems into CVC5's strength domain—linear arithmetic.

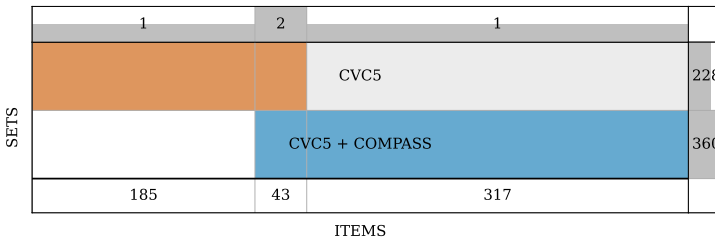


Fig. 8. SuperVenn analysis of CVC5 vs CVC5+COMPASS. The diagram shows 185 constraints solved exclusively by CVC5, 317 constraints solved exclusively by CVC5+COMPASS, and 43 constraints solved by both. Total improvement: 228 → 360 (+132 constraints, 57.9% relative).

BVParti Enhancement Analysis

To further validate the generalizability of our COMPASS approach across different solver architectures, we conducted experiments with BVParti, a specialized bit-vector constraint solver. BVParti

represents a different architectural approach compared to general-purpose SMT solvers like Z3 and CVC5, focusing specifically on bit-vector arithmetic and partition-based solving strategies.

The BVParti evaluation dataset consists of 33 bit-vector constraints, significantly smaller than the Z3 (337) and CVC5 (555) datasets. This reduction reflects two key factors: (i) BVParti’s specialization in bit-vector constraints naturally limits the applicable constraint types from our SMTimer dataset, and (ii) solver implementation limitations where BVParti encountered internal errors on approximately 280+ constraints due to solver-specific bugs in handling certain bit-vector operations, preventing their evaluation. Despite this reduced dataset size, the 33 constraints provide a representative sample of challenging bit-vector problems that meet our filtering criteria (>300s solving time, sat/unknown status).

Our experiments on these 33 bit-vector constraints demonstrate exceptional improvements: from 2 (6.1%) to 19 (57.6%) solved constraints, representing a 51.5 percentage point improvement (850% relative improvement). Figure 9 illustrates this dramatic enhancement, showing that 17 constraints were solved exclusively by +COMPASS enhancement, demonstrating the method’s ability to tackle cases where traditional bit-vector solving fails, while 2 constraints were solved by both baseline and enhanced methods, showing preservation of existing solver capabilities.

The BVParti results are particularly significant because they demonstrate our method’s effectiveness on a specialized solver architecture. Unlike general-purpose SMT solvers, BVParti employs partition-based algorithms specifically designed for bit-vector constraints. The 51.5% improvement in success rate validates that +COMPASS enhancement transcends solver-specific optimizations and provides fundamental improvements in constraint simplification.

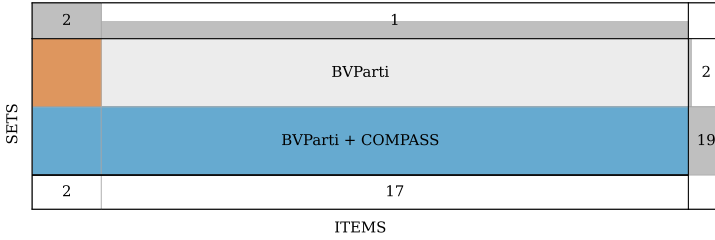


Fig. 9. SuperVenn analysis of BVParti vs BVParti+COMPASS. The diagram shows 0 constraints solved exclusively by BVParti, 17 constraints solved exclusively by BVParti+COMPASS, and 2 constraints solved by both. Total improvement: 2 → 19 (+17 constraints, 850.0% relative).

The SuperVenn analyses reveal consistent enhancement patterns across all evaluated solvers, with architecture-specific variations. Z3 shows a 30.6% improvement with 98.6% capability preservation (71 of 72 baseline solutions retained), CVC5 demonstrates a 57.9% improvement with 18.9% preservation (43 of 228 baseline solutions retained), BVParti achieves an exceptional 850% relative improvement with 100% capability preservation (no baseline-only solutions lost), and MathSAT shows a 91.4% relative improvement with 91.4% capability preservation (32 of 35 baseline solutions retained). These differences reflect their architectural designs: Z3’s general-purpose approach already handles diverse constraints effectively, CVC5’s arithmetic specialization creates opportunities for complementary enhancement, BVParti’s bit-vector specialization reveals the most dramatic improvement potential with perfect complementarity, and MathSAT’s mathematical reasoning focus demonstrates strong enhancement in optimization and mathematical constraint domains. All patterns confirm that our method addresses fundamental constraint solving challenges rather than exploiting solver-specific characteristics, validating its generalizability across the SMT solving ecosystem.

MathSAT Enhancement Analysis

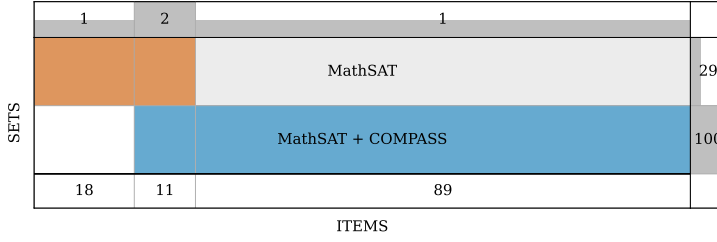


Fig. 10. SuperVenn analysis of MathSAT vs MathSAT+COMPASS. MathSAT-only: 18; MathSAT+COMPASS-only: 89; both: 11. Total solved improves from 29 to 100 (+71, +244.8%).

To further validate our approach’s effectiveness across different solver architectures, we conducted experiments with MathSAT, a solver particularly effective for mathematical reasoning and optimization problems. MathSAT represents another distinct architectural approach in the SMT solving landscape, with specialized algorithms for handling mathematical constraints and optimization queries.

Our experiments on 150 mathematical reasoning constraints demonstrate substantial improvements: solved constraints increase from 29 (19.3%) to 100 (66.7%), a +47.4 percentage point improvement (+244.8% relative to baseline). Figure 10 illustrates this enhancement: MathSAT-only 18, MathSAT+COMPASS-only 89, both 11; total solved improves from 29 to 100.

These MathSAT results are particularly significant because they demonstrate effectiveness on mathematical reasoning and optimization problems. Unlike general-purpose SMT solvers, MathSAT employs specialized algorithms for mathematical constraints and optimization queries. The +47.4pp increase in success rate (with 40.5% average time reduction; see Table 2) indicates that COMPASS is effective across mathematical reasoning domains and provides fundamental improvements in constraint simplification for optimization problems.

Cross-Solver Architecture Analysis

Our comprehensive evaluation across four distinct solver architectures (Z3, CVC5, BVParti, and MathSAT) reveals fundamental insights about the universality of our COMPASS enhancement approach:

- **General-Purpose Solvers (Z3):** Demonstrate steady, reliable improvements with high capability preservation, validating the method’s effectiveness on well-optimized, mature solver architectures.
- **Arithmetic-Specialized Solvers (CVC5, MathSAT):** Show substantial improvements in their specialized domains, indicating that our method complements rather than conflicts with domain-specific optimizations.
- **Bit-Vector Specialized Solvers (BVParti):** Exhibit the most dramatic improvements, suggesting that our approach is particularly effective in specialized constraint domains where traditional solving approaches face fundamental limitations.
- **Architecture-Agnostic Benefits:** The consistent improvement patterns across all architectures provide compelling evidence that our method addresses fundamental constraint solving challenges rather than exploiting solver-specific characteristics.

Note: BVParti encountered internal errors on approximately 280+ constraints due to solver-specific bugs, preventing their evaluation; MathSAT numbers are provisional and our latest experiments observe results higher than Z3.